



REACT

React

100 Interview Questions

QUESTIONS & ANSWERS

From JSX to Server Components, the 100 that matter

Almost every web role tests React, and the interviews have gotten deeper: it is no longer enough to wire up `useState` and call it a day. This guide covers the full range a modern React interview reaches, from fundamentals and hooks to rendering internals, state management, performance, and the React 18 and 19 features interviewers are starting to ask about.

Every answer explains the reasoning a strong candidate would give, with code you can run and the trade-offs behind each pattern. Read it once cover to cover, then use it the night before to reload the whole map.

HOW TO USE THIS GUIDE

- **Explain, do not recite.** After each answer, close the page and say it in your own words. That is what the interview rewards.
- **Type the snippets.** Hooks, dependency arrays, and memoization are muscle memory under pressure.
- **Spend your last hour** on the hooks rules, reconciliation, and re-render sections. That is where most candidates get exposed.

What's inside

Part I	Fundamentals — JSX, components, props, state, events, forms, lists, composition	Q1–20
Part II	Hooks — useState, useEffect, useContext, useRef, useMemo, useCallback, custom hooks, rules	Q21–40
Part III	State & Data — Context, Redux Toolkit, server state, React Query, forms, error boundaries	Q41–60
Part IV	Performance & Rendering — reconciliation, memo, re-renders, code splitting, React 18 batching	Q61–80
Part V	Advanced & Testing — Server Components, React 19, routing, TypeScript, React Testing Library	Q81–100

1 What is React

FUNDAMENTALS

Q What is React and why would you use it?

A React is a JavaScript library for building user interfaces, created and maintained by Meta. Its core value proposition rests on three ideas: it is **declarative**, **component-based**, and it uses a **virtual DOM** to reconcile changes efficiently. Understanding the WHY behind these matters more than memorizing the definition.

Declarative means you describe *what* the UI should look like for a given state, not the step-by-step DOM mutations to get there. Instead of manually calling `document.createElement` and tracking which nodes changed, you write a function of your data and React figures out the minimal set of updates. This eliminates a whole class of bugs where the DOM and your data drift out of sync.

Component-based means you compose your UI from small, reusable, self-contained pieces that manage their own state and logic. Components compose like functions, which makes large applications tractable. Combined with a strong ecosystem (routing, state management, testing) and a huge community, React became a default choice for interactive web apps.

```
function Welcome({ name }) {  
  return <h1>Hello, {name}</h1>;  
}
```

GOTCHA

React is a library, not a full framework. It deliberately leaves routing, data fetching, and global state to the ecosystem or meta-frameworks like Next.js. Interviewers often probe whether you understand that distinction.

2 JSX and Compilation

FUNDAMENTALS

Q What is JSX and how does it compile?

A JSX is a syntax extension to JavaScript that lets you write HTML-like markup inside your code. It is **not** understood by browsers directly; a compiler like Babel or the TypeScript compiler transforms it into plain JavaScript function calls. Knowing what it compiles to demystifies a lot of React's behavior.

Historically, JSX compiled to `React.createElement` calls, which is why you used to need `import React from 'react'` in every file even if you never referenced `React` explicitly. Each element becomes a call that returns a plain JavaScript object describing the node (its type, props, and children) — this object is a React element.

Since React 17, the **new JSX transform** lets the compiler import special functions (`jsx` / `jsxjs`) from `react/jsx-runtime` automatically, so you no longer need to import React just to use JSX. JSX expressions can embed any JavaScript inside curly braces, and attributes use camelCase (e.g. `className`, `onClick`).

```
// This JSX...  
<h1 className="title">Hi</h1>  
  
// ...compiles roughly to:  
React.createElement("h1", { className: "title" }, "Hi");
```

GOTCHA

JSX must return a single root element. That is why fragments exist. Also, because JSX is just function calls, you cannot use statements like `if` directly inside it — only expressions.

3 Function vs Class Components

FUNDAMENTALS

Q What is the difference between function and class components, and why did function components win?

A Both are ways to define a React component. A class component extends `React.Component` and defines a `render` method plus lifecycle methods; a function component is just a function that returns JSX. Since the introduction of Hooks in React 16.8, function components can do everything class components can, and they have become the **de facto standard**.

Function components won for several reasons. They are more concise with less boilerplate — no `this` binding, no constructor. The `this` keyword in classes was a persistent source of bugs, especially around event handlers and stale references. Hooks let you extract and reuse stateful logic (custom hooks) in a way that class-based patterns like higher-order components and render props made awkward.

Hooks also colocate related logic. In classes, one concern (say, a subscription) was split across `componentDidMount` and `componentWillUnmount`; with `useEffect` the setup and cleanup live together. Class components still work and are not deprecated, but new code is written with functions and hooks.

```
// Function component
function Counter() {
  const [n, setN] = useState(0);
  return <button onClick={() => setN(n + 1)}>{n}</button>;
}
```

GOTCHA

There is no hook equivalent to `componentDidCatch` yet — error boundaries still require a class component (or a library wrapper). That is the one thing hooks cannot do.

4 Props

FUNDAMENTALS

Q What are props and what are their key characteristics?

A Props (short for properties) are the inputs to a component — the mechanism a parent uses to pass data down to a child. They are the reason components are reusable: the same component renders differently based on the props it receives. The single most important rule is that props are **read-only**.

A component must never mutate its own props. React enforces a **one-way (top-down) data flow**: data flows from parent to child, never the reverse. If a child needs to communicate upward, the parent passes down a callback function as a prop, and the child calls it. This unidirectional flow makes applications predictable — you can always trace where a piece of data originated.

Props can be any JavaScript value: strings, numbers, objects, arrays, functions, and even other React elements (via `children`). Treating props as immutable is what allows React to reason about when to re-render.

```
function Avatar({ user, size = 40 }) {  
  return <img src={user.avatarUrl} width={size} alt={user.name} />;  
}  
// Parent: <Avatar user={me} size={64} />
```

GOTCHA

Mutating an object or array received via props may appear to work but corrupts shared state and skips re-renders. Always treat props as frozen; create new objects instead of mutating.

5 State and useState

FUNDAMENTALS

Q What is state and how does useState work?

A State is data that a component owns and can change over time, typically in response to user interaction. Unlike props, which come from the parent, state is **local and private** to the component. When state changes, React re-renders the component so the UI reflects the new data. The `useState` hook is how function components declare state.

`useState` returns an array of exactly two items: the current value and a setter function. You call it with the initial value. Crucially, calling the setter does not immediately change the variable — it **schedules a re-render**, and on the next render the hook returns the updated value. State updates are asynchronous and may be batched together for performance.

When the new state depends on the previous state, use the **functional updater** form so you always work from the latest value rather than a stale closure. State must be treated as immutable: replace objects and arrays rather than mutating them, or React will not detect the change.

```
const [count, setCount] = useState(0);

// Correct: functional update avoids stale state
setCount(prev => prev + 1);

// Objects: create a new object, do not mutate
setUser(prev => ({ ...prev, name: "Ada" }));
```

GOTCHA

Calling `setCount(count + 1)` three times in one event handler still only increments by one, because all three read the same stale `count`. The functional form `setCount(c => c + 1)` fixes this.

6 Props vs State

FUNDAMENTALS

Q What is the difference between props and state?

A This is one of the most common interview questions because it tests whether you understand React's data model. Both are plain JavaScript objects holding data that influences render output, but they serve opposite roles. The key distinction is **ownership and mutability**.

Props are passed *into* a component from its parent and are read-only from the component's perspective — the component cannot change them. State is owned and managed *by* the component itself and can be changed with its setter, triggering a re-render. Props enable configuration and communication between components; state enables interactivity within a component.

A useful heuristic: if data comes from above and never changes locally, it is a prop. If it changes over time due to something inside the component (user input, timers, fetches), it is state. Notably, one component's state is frequently passed down as another component's props — that is the whole flow of a React app.

```
// Parent owns state, passes it down as a prop
function Parent() {
  const [text, setText] = useState("");
  return <Child value={text} />; // value is a prop to Child
}
```

GOTCHA

Do not copy props into state just to store them (e.g. `useState(props.value)`) unless you intend to fork it. Doing so causes the "stale state" bug where the copy ignores later prop updates.

7 What Triggers a Render

FUNDAMENTALS

Q What triggers a component to render or re-render?

A Understanding rendering is foundational to reasoning about performance. A "render" is React calling your component function to compute what the UI should look like. There are exactly two reasons a component renders: its **initial mount**, or a subsequent **re-render** triggered by a state change.

A re-render happens when: (1) the component's own state changes via a setter, (2) its parent re-renders, or (3) a context value it consumes changes. Importantly, when a component re-renders, React by default re-renders **all of its descendants** regardless of whether their props changed. This surprises people — a parent's state change cascades down the tree.

Rendering is not the same as updating the DOM. React calls your function, produces a new virtual DOM tree, and then *reconciles* it against the previous one. Only the actual differences are committed to the real DOM. So a re-render is often cheap even when it does not change any pixels. Memoization (`React.memo` , `useMemo`) can skip re-rendering subtrees when it matters for performance.

```
// A parent re-render re-renders Child too, even
// though Child received no changed props.
const Child = React.memo(function Child({ label }) {
  return <span>{label}</span>;
}); // memo skips re-render if props are equal
```

GOTCHA

Setting state to the same value it already has (by `Object.is` comparison) causes React to bail out and skip the re-render. But passing a new object literal with the same contents is a different reference and will re-render.

8 The Virtual DOM

FUNDAMENTALS

Q What is the virtual DOM and why does it exist?

A The virtual DOM (VDOM) is a lightweight, in-memory JavaScript representation of the real DOM. When you render, React builds a tree of plain objects describing the UI. On an update, it builds a new tree and **diffs** it against the previous one — a process called reconciliation — then applies only the minimal set of changes to the real DOM.

It exists because **direct DOM manipulation is expensive**. Touching the real DOM triggers layout, style recalculation, and repaint. Doing many small mutations, or blowing away and rebuilding large sections, is slow. The VDOM lets React batch and minimize real DOM operations: instead of you hand-optimizing updates, React computes the difference and updates surgically.

Equally important, the VDOM enables the **declarative programming model**. You describe the target UI for the current state, and React handles the transition from the old UI to the new one. The diffing algorithm uses heuristics (same element type, and keys for lists) to stay roughly $O(n)$ rather than the naive $O(n^3)$ of general tree diffing.

```
// A VDOM node is just a plain object:
{
  type: "button",
  props: { className: "btn", children: "Save" }
}
```

GOTCHA

The VDOM is not automatically "faster" than optimized vanilla DOM code — it trades a little overhead for developer productivity and predictable performance. Interviewers like candidates who acknowledge this nuance rather than parroting "VDOM is fast."

9 Handling Events

FUNDAMENTALS

Q How does event handling work in React?

A In React you attach event handlers directly in JSX using camelCase props like `onClick`, `onChange`, and `onSubmit`, passing a function (not a string). Under the hood React does not attach a listener to every node; instead it uses a **synthetic event system** with event delegation at the root container.

A **SyntheticEvent** is React's cross-browser wrapper around the native event. It normalizes behavior so your code works consistently across browsers, and exposes the familiar API (`e.preventDefault()`, `e.stopPropagation()`, `e.target`). Since React 17, events are attached to the React root rather than the document, improving compatibility when embedding React in larger pages.

You pass a *reference* to the handler, not a call: `onClick={handleClick}`, not `onClick={handleClick()}` (the latter calls it during render). To pass arguments, wrap it in an arrow function. Be mindful that creating a new arrow function each render is usually fine, but can matter for memoized children.

```
function Button({ id, onRemove }) {
  return (
    <button onClick={(e) => {
      e.preventDefault();
      onRemove(id);
    }}>Remove</button>
  );
}
```

GOTCHA

Returning `false` from a React handler does *not* prevent default behavior like it does in inline HTML. You must call `e.preventDefault()` explicitly.

10 Conditional Rendering

FUNDAMENTALS

Q What are the common patterns for conditional rendering?

A Because JSX is just JavaScript expressions, you render conditionally using ordinary JS. There is no special template syntax. The main patterns are the ternary operator, logical `&&` short-circuiting, early returns, and assigning JSX to a variable ahead of the return.

Use the **ternary** (`cond ? a : b`) when you choose between two branches. Use **logical AND** (`cond && <X />`) to render something or nothing. For complex logic, compute the element in a variable or return early from the function — this keeps JSX readable. Returning `null` from a component renders nothing.

The classic bug with `&&` is that if the left operand is `0` (a number), React will render the `0` rather than nothing, because `0` is falsy but is still a valid React child. Guard with a real boolean, e.g. `items.length > 0 && ...`.

```
function Cart({ items }) {  
  if (items.length === 0) return <p>Empty</p>;  
  return (  
    <div>  
      {items.length > 0 && <span>{items.length} items</span>}  
    </div>  
  );  
}
```

GOTCHA

`{count && <Badge />}` renders a literal `0` when `count` is zero. Convert to a boolean: `{!!count && <Badge />}` or `{count > 0 && ...}`.

11 Lists and Keys

FUNDAMENTALS

Q How do you render lists and why do keys matter?

A You render lists by mapping an array of data to an array of elements, typically with `Array.prototype.map`. React can render an array of elements directly. Each element in such a list must have a **unique key prop** among its siblings, and this key is central to correct and efficient updates.

Keys give each item a stable identity across renders. During reconciliation, React uses keys to match elements between the old and new lists so it can tell which items were added, removed, moved, or merely reordered — and reuse DOM nodes and component state accordingly. Without stable keys, React falls back to index-based matching, which can misassociate state and cause subtle bugs.

Use a stable, unique ID from your data as the key. Avoid using the array **index** as a key when the list can reorder, filter, or have items inserted, because the index of an item changes and React will associate the wrong state with the wrong item. Index keys are acceptable only for static lists that never change order.

```
function List({ users }) {  
  return (  
    <ul>  
      {users.map((u) => (  
        <li key={u.id}>{u.name}</li>  
      ))}  
    </ul>  
  );  
}
```

GOTCHA

Keys must be unique among *siblings*, not globally. They are also not accessible as a prop inside the component — reading `props.key` returns undefined. Pass the id separately if you need it.

12 Controlled vs Uncontrolled

FUNDAMENTALS

Q What is the difference between controlled and uncontrolled components?

A This distinction applies to form inputs. In a **controlled component**, React state is the single source of truth: the input's value is driven by state via the `value` prop, and every keystroke fires `onChange` to update that state. In an **uncontrolled component**, the DOM itself holds the value, and you read it when needed using a ref.

Controlled inputs give you full control: you can validate, transform, or conditionally disable input on every keystroke, and the state always reflects what the user sees. This is the recommended default. The trade-off is more code and a re-render per keystroke. Uncontrolled inputs are simpler for basic cases and integrate more easily with non-React code, at the cost of losing React-driven control.

An uncontrolled input can still have an initial value via `defaultValue` (or `defaultChecked`), which sets the DOM's starting value without controlling it thereafter. Mixing a `value` without an `onChange` makes the field read-only and triggers a warning.

```
// Controlled
const [name, setName] = useState("");
<input value={name} onChange={(e) => setName(e.target.value)} />

// Uncontrolled
const ref = useRef();
<input defaultValue="hi" ref={ref} />
```

GOTCHA

Switching an input from uncontrolled to controlled (e.g. `value` starts as `undefined` then becomes a string) triggers React's "changing an uncontrolled input to be controlled" warning. Initialize state to `""`, not `undefined`.

13 Forms in React

FUNDAMENTALS

Q How do you build forms with controlled inputs?

A A form in React is typically a set of controlled inputs whose values live in state, plus an `onSubmit` handler on the `<form>` element. On submit you call `e.preventDefault()` to stop the browser's default full-page reload, then process the state values (validate, send to an API, etc.).

For multiple fields, a common pattern is a single state object and a shared change handler that keys off the input's `name` attribute. This scales better than one `useState` per field. Each input reads its value from state and writes back through `onChange`, keeping the form fully in sync with React state.

Attaching the submit handler to the form (rather than the button's click) ensures the form also submits on Enter key, which is important for accessibility. Modern React (19+) also adds form `action` support and hooks like `useActionState`, but the controlled-input pattern remains the fundamental technique to know.

```
function SignupForm() {
  const [form, setForm] = useState({ email: "", pass: "" });
  const onChange = (e) =>
    setForm((f) => ({ ...f, [e.target.name]: e.target.value }));
  const onSubmit = (e) => { e.preventDefault(); /* submit */ };
  return (
    <form onSubmit={onSubmit}>
      <input name="email" value={form.email} onChange={onChange} />
    </form>
  );
}
```

GOTCHA

Computed property names (`[e.target.name]`) require every input to have a matching `name` attribute. Forgetting it makes the key `undefined` and silently overwrites the wrong field.

14 Lifting State Up

FUNDAMENTALS

Q What does "lifting state up" mean and when do you do it?

A Lifting state up is the pattern of moving shared state to the **closest common ancestor** of the components that need it. When two sibling components must stay in sync or read the same data, you cannot pass data sideways between them — data only flows down. So you hoist the state into their shared parent and pass it down as props.

The parent owns the state and passes both the value and a setter (or a more specific callback) down to each child. Children read the value via props and request changes by invoking the callback. This keeps a **single source of truth**: there is exactly one place the data lives, which prevents the two copies from diverging.

This is the direct consequence of React's one-way data flow. When lifting causes deeply nested prop-passing ("prop drilling"), that is the signal to reach for Context or a state-management library — but lifting state up is the first, simplest tool and often sufficient.

```
function Parent() {
  const [temp, setTemp] = useState(0);
  return (
    <>
      <Input value={temp} onChange={setTemp} />
      <Display value={temp} />
    </>
  );
}
```

GOTCHA

Do not lift state higher than necessary. Over-lifting makes the top component a "god component" and causes broad re-renders. Keep state as local as it can be while still shared where needed.

15 Composition vs Inheritance

FUNDAMENTALS

Q Why does React favor composition over inheritance?

A React strongly recommends **composition** over class inheritance for reusing code between components. Rather than having a component extend another to share behavior, you compose components by nesting them and passing content through props — most notably the special `children` prop.

The `children` prop contains whatever JSX you place between a component's opening and closing tags, letting you build generic "container" components (cards, modals, layouts) that do not know what they will hold. This is more flexible than inheritance: a component can accept multiple slots as props (e.g. `header`, `footer`), compose freely, and avoid rigid class hierarchies.

Meta's team has stated they found no use cases where component inheritance offered advantages over composition and props. If you want to reuse non-visual logic, extract a **custom hook** or a plain JavaScript function rather than a base class. Composition keeps components loosely coupled and easy to reason about.

```
function Card({ title, children }) {  
  return (  
    <section className="card">  
      <h2>{title}</h2>  
      <div>{children}</div>  
    </section>  
  );  
}  
// <Card title="Hi"><p>Body</p></Card>
```

GOTCHA

`children` can be anything — a string, an array, a function (render prop), or nothing. Do not assume it is always a single element; use `React.Children` utilities if you must inspect or map over it.

16 Fragments

FUNDAMENTALS

Q What are Fragments and why are they needed?

A A component must return a single root element, because JSX compiles to a single function call that returns one node. When you want to return several sibling elements without introducing an extra wrapper in the DOM, you use a **Fragment**. It groups children without adding any node to the rendered output.

The long form is `<React.Fragment>...</React.Fragment>`, and the shorthand is the empty tag `<>...</>`. Fragments matter because a wrapping `<div>` can break CSS layouts like flexbox and grid, or produce invalid HTML — for example, a `<td>` must be a direct child of a `<tr>`, so you cannot wrap table cells in a div.

When rendering a list of fragments you need keys, and only the explicit

`<React.Fragment key={...}>` form accepts a key — the shorthand `<>` cannot take props. Fragments keep the DOM clean and semantic.

```
function Row() {
  return (
    <>
      <td>Name</td>
      <td>Email</td>
    </>
  );
}
```

GOTCHA

The shorthand `<>` cannot receive a `key`. Inside a `.map` you must switch to `<React.Fragment key={id}>`.

17 React.StrictMode

FUNDAMENTALS

Q What is React.StrictMode and what does it do?

A `<StrictMode>` is a development-only wrapper component that activates extra checks and warnings for its subtree. It renders no visible UI and has **zero effect in production builds** — it exists purely to help you catch bugs early and write code that will be compatible with future React features.

Its most noticeable behavior is that, in development, React **intentionally double-invokes** certain functions — component render bodies, state updater functions, and effect setup/cleanup. This is deliberate: it surfaces components that are not pure or effects that lack proper cleanup. If double-rendering breaks your component, it has a hidden side effect that should be fixed.

StrictMode also warns about deprecated APIs and legacy patterns (like the old string refs or unsafe lifecycle methods). Because the double-invocation only happens in development, the mysterious "my effect runs twice" behavior developers report is StrictMode doing its job — the fix is to make effects idempotent with cleanup, not to remove StrictMode.

```
import { StrictMode } from "react";

createRoot(el).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

GOTCHA

The double-render trips people up when they put side effects (fetches, mutations) directly in the render body or without cleanup in an effect. Do not disable StrictMode to hide this — it is a signal of a real impurity.

18 Default Props and Validation

FUNDAMENTALS

Q How do you set default prop values and validate props?

A For function components, the idiomatic way to provide defaults is **default parameter values** via destructuring: `function Btn({ size = "md" })`. This is cleaner than the legacy `Component.defaultProps` static property, which is deprecated for function components in React 19.

For runtime validation, the classic tool is the `prop-types` package. You attach a `Component.propTypes` object declaring the expected type of each prop; in development, React warns in the console when a prop is missing or has the wrong type. PropTypes checks are stripped in production and add no runtime cost there.

In modern codebases, **TypeScript** has largely replaced PropTypes. TypeScript validates props at *compile time* using an interface or type for the props, catching mismatches before the code ever runs and providing editor autocompletion. This is strictly more powerful than PropTypes' runtime-only checking, which is why new projects overwhelmingly choose TypeScript.

```
import PropTypes from "prop-types";

function Badge({ label, count = 0 }) {
  return <span>{label}: {count}</span>;
}

Badge.propTypes = {
  label: PropTypes.string.isRequired,
  count: PropTypes.number,
};
```

GOTCHA

PropTypes only runs in development and only warns — it never blocks rendering. It cannot replace TypeScript's compile-time guarantees, and mixing both is redundant. Pick one (usually TypeScript).

19 Styling Approaches

FUNDAMENTALS

Q What are the main ways to style React components?

A React is unopinionated about styling, so there are several approaches, each with trade-offs. The most basic is the `className` prop pointing at plain CSS classes (React uses `className` instead of `class` because `class` is a reserved JavaScript keyword).

CSS Modules scope class names locally to a component by hashing them at build time, preventing global name collisions — you import the styles object and reference `styles.button`. **Inline styles** use the `style` prop, which takes a JavaScript object with camelCased properties (`backgroundColor` , not `background-color`); good for dynamic values but limited (no media queries or pseudo-selectors).

CSS-in-JS libraries (styled-components, Emotion) let you write component-scoped styles in JS with dynamic props. **Utility-first** frameworks like Tailwind CSS apply pre-built classes directly in markup and are hugely popular. The right choice depends on team preference, performance needs (runtime vs zero-runtime), and tooling.

```
import styles from './Button.module.css';

function Button() {
  return (
    <button
      className={styles.primary}
      style={{ marginTop: 8 }}
    >Go</button>
  );
}
```

GOTCHA

The `style` prop takes an *object*, not a string, hence the double braces: the outer `{}` is JSX interpolation and the inner `{}` is the object literal. Numeric values default to pixels for most properties.

20 Pure Components

FUNDAMENTALS

Q What does it mean that components should be pure functions of their props and state?

A React expects your component render logic to be **pure**: given the same props and state, it must return the same JSX, and it must not produce side effects during rendering. A pure function does not mutate anything outside its scope, does not mutate its inputs, and does not perform I/O while computing its result.

Purity is what makes React's model work. Because rendering is side-effect-free, React is free to call your component whenever it wants, to bail out of renders, to render in the background, or (in Concurrent features) to pause and restart a render. If your render had side effects, these optimizations would produce inconsistent behavior — hence StrictMode's double-render that flushes out impurity.

Side effects belong outside render: event handlers (which run in response to user actions) and the `useEffect` hook (which runs after commit). The `React.memo` higher-order component (the modern successor to the class `PureComponent`) leverages purity by shallow-comparing props and skipping re-render when they are unchanged.

```
// Pure: same props => same output, no mutation
function Price({ amount, rate }) {
  const total = amount * rate; // derive, do not mutate
  return <span>${total.toFixed(2)}</span>;
}
```

GOTCHA

Mutating a prop or a module-level variable during render is impure even if it "works."

`React.memo` and Concurrent rendering assume purity — impure components can render stale or duplicated output.

21 What Hooks Are

HOOKS

Q What are React Hooks and why were they introduced?

A Hooks are functions that let you "hook into" React state and lifecycle features from function components. Before Hooks (React 16.8, 2019), only class components could hold local state or run lifecycle logic like `componentDidMount`. Hooks were introduced to solve real pain points, not just to make functions fancier, so understanding the WHY matters more than memorizing the list.

The core problem was **reusing stateful logic**. In classes, patterns like render props and higher-order components let you share behavior, but they forced you to restructure your component tree and produced "wrapper hell". Hooks let you extract stateful logic into plain reusable functions (custom hooks) without changing your component hierarchy at all.

A second problem was that related code got split across lifecycle methods: a subscription's setup lived in `componentDidMount` and its teardown in `componentWillUnmount`, while unrelated concerns were jammed together. Hooks organize code by what it does, not by which lifecycle it fires in. They also sidestep JavaScript's confusing `this` binding and let compilers optimize function components more easily.

```
function Counter() {  
  const [count, setCount] = useState(0);  
  return <button onClick={() => setCount(count + 1)}>{count}</button>;  
}
```

GOTCHA

Hooks did not deprecate classes; class components still work. But new code, and nearly all library guidance, is written for function components with Hooks.

22 The Rules of Hooks

HOOKS

Q What are the Rules of Hooks, and why do they exist?

A There are two rules: (1) only call Hooks at the top level of a component or another Hook, never inside loops, conditions, or nested functions; and (2) only call Hooks from React function components or custom Hooks, not from regular JavaScript functions. The WHY comes down to how React tracks Hook state internally.

React does not identify a Hook by a name or key you pass. It relies purely on **call order**. On each render, React walks a linked list of Hook "slots" in the exact sequence the Hooks are called. The first `useState` is slot 0, the second is slot 1, and so on. If you call a Hook conditionally, the order can shift between renders, so React associates the wrong state with the wrong Hook, corrupting everything downstream.

The second rule ensures Hooks only run where React has an active render context to attach that linked list to. Calling a Hook from an ordinary helper function means there is no component instance to own the state. The ESLint plugin `eslint-plugin-react-hooks` enforces both rules statically and also checks dependency arrays, so keep it enabled.

```
// WRONG: conditional Hook breaks slot ordering
if (isLoggedIn) {
  const [name, setName] = useState("");
}
// RIGHT: call unconditionally, branch on the value
const [name, setName] = useState("");
```

GOTCHA

Early `return` statements are a sneaky violation: any Hook after a conditional `return` may be skipped on some renders. Put all Hook calls before any early return.

23 useState in Depth

HOOKS

Q Explain useState, including functional updates and lazy initialization.

A `useState` returns a stateful value and a setter. Calling the setter schedules a re-render; it does not mutate the current variable synchronously. The value is preserved across renders because React stores it in the Hook slot, not in the local variable. Understanding two subtleties, functional updates and lazy init, separates a shallow answer from a strong one.

Functional updates matter when the next state depends on the previous state. If you call `setN(n + 1)` multiple times in one event, each reads the same stale `n` from that render's closure, so they collapse into one increment. Passing a function, `setN(prev => prev + 1)`, tells React to apply updates in sequence against the latest queued value, so three calls reliably add three.

Lazy initialization addresses expensive initial values. The argument to `useState` is evaluated on every render even though it is only used on the first, wasting work. Passing a function, `useState(() => expensiveInit())`, makes React call it exactly once, on mount. Also remember state updates are batched and that setting state to the same value (by `Object.is`) can bail out of a re-render.

```
const [count, setCount] = useState(() => readFromStorage());
setCount(prev => prev + 1); // functional update
setCount(prev => prev + 1); // both apply: +2 total
```

GOTCHA

State updates are not merged like `this.setState` in classes. If state is an object, you must spread the old value yourself: `setUser(u => ({ ...u, name }))`.

24 useEffect Basics

HOOKS

Q What is `useEffect` for, and how does the dependency array work?

A `useEffect` lets you run **side effects** after render: things that reach outside React's pure render flow, such as network requests, subscriptions, timers, logging, or manually touching the DOM. React deliberately defers effects until after the browser has painted, so rendering stays fast and effects see a committed screen. The WHY: render must be pure, so anything with an external consequence belongs in an effect.

The dependency array controls when the effect re-runs. React compares each dependency with the previous render's value using `Object.is`. With **no array**, the effect runs after every render. With an **empty array** `[]`, it runs only once after mount. With `[a, b]`, it re-runs whenever `a` or `b` changed since the last render.

The mental model is not "run on mount/update" but "synchronize with these values." Every value from the component scope that the effect reads (props, state, functions) should be in the array. Effects are meant to keep some external system in sync with your React state; when a dependency changes, React re-synchronizes by cleaning up and running again.

```
useEffect(() => {  
  const id = setInterval(tick, 1000);  
  return () => clearInterval(id); // cleanup  
}, [tick]);
```

GOTCHA

Not every bit of logic needs an effect. If you can compute a value during render from existing props/state, do that instead; effects for derived data cause extra renders and bugs.

25 Effect Cleanup & Timing

HOOKS

Q How does effect cleanup work, and when do effects run and re-run?

A An effect may return a cleanup function. React runs it before the next execution of that same effect and once more when the component unmounts. The purpose is to undo whatever the effect set up: unsubscribe, clear a timer, abort a request, remove an event listener. Skipping cleanup is the classic source of memory leaks and duplicate subscriptions.

The precise timing on an update is: React renders, commits the new DOM, then before running the effect again it runs the **previous** render's cleanup, then runs the new effect. So on every dependency change you get "clean up the old, set up the new." This is why an effect that subscribes to `userId` correctly detaches from the old user before attaching to the new one.

Effects fire after paint (asynchronously), so the user never sees an intermediate state flash from an effect. In development with Strict Mode, React intentionally **mounts, unmounts, and remounts** each component once, running your effect twice, to surface missing cleanup. Correctly written effects (setup fully reversed by cleanup) are resilient to this double invocation.

```
useEffect(() => {  
  const handler = () => setWidth(window.innerWidth);  
  window.addEventListener("resize", handler);  
  return () => window.removeEventListener("resize", handler);  
}, []);
```

GOTCHA

Strict Mode's double-invoke only happens in development, never production. If code breaks under it, the effect has a real cleanup bug; do not "fix" it by disabling Strict Mode.

26 Common useEffect Mistakes

HOOKS

Q What are the most common useEffect mistakes, like missing deps and infinite loops?

A The frequent mistakes are: omitting dependencies, creating infinite loops, and overusing effects for logic that belongs in render or event handlers. Each stems from misunderstanding that an effect re-synchronizes whenever its dependencies change.

Missing dependencies cause stale values: the effect closes over an old prop or state and never updates. The lint rule flags these; the correct fix is almost never to silence the rule but to include the dependency, or to remove the need for it (functional updates, moving a function inside the effect, or wrapping it in `useCallback`). Suppressing the warning tends to hide real bugs.

Infinite loops happen when an effect updates state that is (directly or indirectly) one of its dependencies, or when a dependency is a new object/array/function created every render. Each render creates a fresh reference, so `Object.is` sees a change, the effect re-runs, updates state, re-renders, and repeats forever. Fixes: use functional updates, memoize the dependency, or move object creation out. Also avoid using effects to compute derived state you could calculate during render.

```
// Infinite loop: options is a new object each render
useEffect(() => { fetchData(options); }, [options]);
// Fix: memoize or depend on primitive fields
const options = useMemo(() => ({ id }), [id]);
```

GOTCHA

Responding to a user event? Put the logic in the event handler, not an effect. Effects are for synchronizing with external systems, not for reacting to interactions.

27 useContext

HOOKS

Q Explain useContext and the Context pattern.

A Context provides a way to pass data through the component tree without threading props manually at every level ("prop drilling"). You create a context with `createContext`, wrap a subtree in its `Provider` with a `value`, and any descendant reads that value with `useContext`. It is ideal for broadly needed, relatively stable data like theme, current user, or locale.

The WHY: prop drilling couples intermediate components to data they do not use, purely to forward it. Context removes that intermediary burden. A consumer subscribes directly to the nearest matching Provider above it, so adding or removing middle layers no longer requires rewiring props.

The key performance caveat: **every consumer re-renders when the Provider's value changes**, by `Object.is` comparison. If you pass an inline object as `value`, it is a new reference every render and forces all consumers to re-render. Memoize the value, and split unrelated data into separate contexts so a change to one does not re-render consumers of the other. Context is for sharing, not a full state manager.

```
const ThemeContext = createContext("light");

function Toolbar() {
  const theme = useContext(ThemeContext);
  return <div className={theme}/>;
}
```

GOTCHA

A component reading context re-renders whenever the value changes even if it only uses one field. For high-frequency updates, prefer a store (Redux, Zustand) or split contexts finely.

28 useRef

HOOKS

Q What is useRef used for, both DOM refs and mutable values?

A `useRef` returns a mutable object `{ current: initialValue }` that persists for the component's full lifetime. It has two distinct uses: accessing DOM nodes, and holding any mutable value that should survive renders without triggering one. The defining property: **mutating `ref.current` does not cause a re-render.**

For **DOM access**, you attach the ref to an element via the `ref` attribute; after commit, `ref.current` points to the DOM node, letting you focus an input, measure size, or integrate a non-React library. React sets the node during commit and clears it to `null` on unmount.

For **mutable instance values**, a ref is like a class instance field: a timer ID, a previous value, a flag indicating whether the component is mounted, or the latest value of a prop for a stale-closure workaround. Because writing it does not re-render, never read a ref during rendering to compute output; that makes render impure and unpredictable. Use state when a change should be reflected on screen; use a ref when it should not.

```
const inputRef = useRef(null);
useEffect(() => { inputRef.current?.focus(); }, []);
return <input ref={inputRef} />;
```

GOTCHA

Do not initialize a ref with an expensive call: `useRef(new Heavy())` constructs on every render (the arg is evaluated each time), even though only the first result is kept. Guard it or use lazy assignment.

29 useMemo

HOOKS

Q What does useMemo memoize, and when should you use it?

A `useMemo` caches the **result** of a computation between renders. You give it a function and a dependency array; React runs the function and stores its return value, re-running it only when a dependency changes. Otherwise it returns the cached value. The purpose is to skip expensive recalculations and, just as importantly, to preserve object/array reference identity across renders.

Two legitimate reasons to reach for it: first, an **expensive calculation** (filtering a large list, heavy math) that would otherwise run every render. Second, **referential stability**: if a memoized value is passed to a `React.memo` child or used as another Hook's dependency, keeping the same reference prevents unnecessary re-renders or effect re-runs.

The WHY behind restraint: memoization is not free. It costs memory and a dependency comparison on every render, and the cache can be dropped by React at any time, so it is an optimization, not a guarantee. For cheap computations the overhead outweighs the benefit. Profile first; do not wrap everything reflexively. Note the compiler ("React Compiler") can automate much of this in newer setups.

```
const sorted = useMemo(  
  () => items.slice().sort(compare),  
  [items]  
);
```

GOTCHA

`useMemo` memoizes a value; it does not memoize the component's render. If the component re-renders for another reason, the memoized value is reused but the surrounding JSX still rebuilds.

30 useCallback

HOOKS

Q What is useCallback, and how does it differ from useMemo?

A `useCallback` memoizes a **function** reference across renders. Without it, every render creates a brand-new function object, even if the body is identical.

`useCallback(fn, deps)` returns the same function instance until a dependency changes. It is literally a specialization of `useMemo`: `useCallback(fn, deps)` equals `useMemo(() => fn, deps)`.

The difference in one line: `useMemo` caches the **value a function returns**; `useCallback` caches the **function itself**. Use `useMemo` for a computed result, use `useCallback` for a callback you will pass down.

The WHY: a stable function reference only matters when identity is observed. That happens when the function is passed to a `React.memo`-wrapped child (so the child does not re-render on a new reference) or listed as a dependency of another effect/memo (so it does not re-fire). If a function is only called locally and never handed off, wrapping it in `useCallback` adds cost with no benefit. Remember its own dependencies must be complete, or you reintroduce stale closures.

```
const handleClick = useCallback(
  () => onSelect(id),
  [onSelect, id]
);
// stable ref => memoized child skips re-render
```

GOTCHA

`useCallback` only helps if the consumer actually compares the reference. Passing a memoized callback to a plain, non-memoized child changes nothing; that child re-renders with its parent regardless.

31 React.memo

HOOKS

Q What is React.memo and how does it pair with useCallback?

A `React.memo` is a higher-order component that memoizes a component's rendered output. It shallow-compares the new props with the previous props and, if they are equal, skips re-rendering, reusing the last result. Its purpose is to prevent a child from re-rendering when its parent re-renders but the child's props have not actually changed.

The catch is **shallow comparison** combined with referential inequality. If a parent passes an inline object, array, or function as a prop, that prop is a new reference on every parent render, so the shallow check always reports "changed" and `React.memo` is defeated. This is exactly where `useCallback` and `useMemo` come in: they preserve stable references for function and object props so the memo comparison can succeed.

So the trio works together: `React.memo` on the child skips work when props are equal; `useCallback` / `useMemo` in the parent keep those props referentially stable. Missing either half breaks the optimization. Use it where a component is genuinely expensive or re-renders often with the same props; sprinkling it everywhere adds comparison overhead for no gain.

```
const Row = React.memo(function Row({ onPick, label }) {  
  return <li onClick={onPick}>{label}</li>;  
});
```

GOTCHA

You can pass a custom comparison as the second argument, but returning `true` means "props are equal, skip render" which is the opposite polarity of `shouldComponentUpdate`. Easy to invert by mistake.

32 Custom Hooks

HOOKS

Q What are custom Hooks, and how do they help reuse logic?

A A custom Hook is a JavaScript function whose name starts with `use` and that calls other Hooks. It lets you extract stateful, reusable logic out of a component into a shareable function. This is the payoff of the whole Hooks model: sharing behavior without wrapper components or changing the tree, which HOCs and render props could not do cleanly.

The crucial insight is that custom Hooks share **logic, not state**. Each call to a custom Hook gets its own fully independent state and effects. If two components both call `useFormInput()`, they do not share a value; each instance is isolated, just as calling `useState` twice yields two separate states.

The `use` prefix is not cosmetic: it signals to the linter that Rules of Hooks apply, so it can verify the function is called correctly and that its internal Hooks obey the ordering rules. Good custom Hooks have a clear single responsibility, return exactly what callers need (a value, a tuple, or an object), and compose freely, one custom Hook calling another.

```
function useToggle(initial = false) {  
  const [on, setOn] = useState(initial);  
  const toggle = useCallback(() => setOn(v => !v), []);  
  return [on, toggle];  
}
```

GOTCHA

A function that does not call any Hook does not need the `use` prefix, and misusing the prefix on a plain helper misleads the linter. Reserve `use` for functions that actually use Hooks.

33 useReducer

HOOKS

Q What is useReducer, and when should you prefer it over useState?

A `useReducer` manages state through a reducer function: `(state, action) => newState`. You call `dispatch(action)`, React runs the reducer with the current state and that action, and the return value becomes the next state. It is an alternative to `useState` for more complex state logic, following the same pattern Redux popularized but built into React.

Prefer it when the **next state depends on intricate rules**, when multiple sub-values change together, or when several actions update the same state in different ways.

Centralizing transitions in one pure reducer makes them easier to read, test in isolation, and reason about than a scatter of `setState` calls. It also shines when the update logic is complex enough that keeping it out of the component improves clarity.

A practical performance and correctness benefit: `dispatch` has a **stable identity** and never changes across renders, so you can safely pass it to memoized children or omit it from dependency arrays. It also cooperates well with Context, letting deeply nested components dispatch actions without prop drilling. Both Hooks are equivalent in power; the choice is about which makes the logic clearer.

```
function reducer(state, action) {
  switch (action.type) {
    case "inc": return { count: state.count + 1 };
    default: return state;
  }
}
const [state, dispatch] = useReducer(reducer, { count: 0 });
```

GOTCHA

Reducers must be pure: no fetches, no mutations, no side effects. Returning the exact same state object (not a new one) lets React bail out of re-rendering.

34 useLayoutEffect vs useEffect

HOOKS

Q How does useLayoutEffect differ from useEffect?

A Both run side effects, but at different times relative to the browser paint. `useEffect` runs **asynchronously after** the browser has painted the new frame.

`useLayoutEffect` runs **synchronously after** the DOM mutations but **before** the browser paints. That timing distinction is the entire point.

Use `useLayoutEffect` when you must read layout (measure a node's size or position) and then synchronously re-render or mutate the DOM before the user sees anything. Because it blocks paint, React applies your change in the same frame, so the user never observes a flicker or an intermediate position. The classic case is measuring a tooltip and repositioning it before it appears.

The trade-off: it blocks painting, so heavy work there hurts perceived performance. The default should be `useEffect`; reach for the layout variant only to prevent a visual flash caused by reading and adjusting layout. Note that on the server, `useLayoutEffect` does not run and React warns about it; guard it or prefer `useEffect` for isomorphic code.

```
useLayoutEffect(() => {
  const { height } = ref.current.getBoundingClientRect();
  setHeight(height); // applied before paint, no flicker
}, []);
```

GOTCHA

If you see content flash at the wrong size or position for one frame, switching that measurement effect from `useEffect` to `useLayoutEffect` usually fixes it.

35 Stale Closures

HOOKS

Q What are stale closures in Hooks, and how do you fix them?

A A stale closure is when a function captures variables from a past render and keeps using those old values after state has moved on. Because each render creates fresh functions bound to that render's props and state, a callback stored somewhere (an interval, an event listener, an effect with missing deps) may "freeze" the values it closed over. This is a consequence of JavaScript closures interacting with React's render-per-state model.

The classic example is a `setInterval` set up once in an empty-dependency effect that reads `count`. The interval callback captured `count` as 0 forever, so it always logs 0 even as the state increases, because the effect never re-ran to capture a newer value.

Fixes, in order of preference: (1) use a **functional updater** so you never read the stale value at all, `setCount(c => c + 1)`; (2) include the value in the dependency array so the effect re-subscribes with fresh values; or (3) store the latest value in a **ref** and read `ref.current` inside the callback, since refs are mutable and always current. The lint rule catches most cases before they bite.

```
useEffect(() => {
  const id = setInterval(() => {
    setCount(c => c + 1); // no stale read
  }, 1000);
  return () => clearInterval(id);
}, []);
```

GOTCHA

Do not "solve" a stale closure by disabling the exhaustive-deps lint rule. That silences the symptom while leaving the frozen value in place.

36 Data Fetching in Effects

HOOKS

Q How do you fetch data with `useEffect`, and what are the pitfalls?

A A basic pattern runs a fetch inside `useEffect` keyed on the query dependencies, stores the result in state, and renders it. It works, but doing it correctly requires handling several pitfalls that naive versions ignore: race conditions, cleanup, loading/error states, and duplicate requests.

The biggest hazard is a **race condition**. If the dependency changes quickly, an earlier request may resolve after a later one and overwrite fresh data with stale results. The fix is to track whether the effect is still current, via a boolean flag in the cleanup, or better an `AbortController` that cancels the outdated request. You should also handle errors, set a loading flag, and remember Strict Mode will run the effect twice in development.

Because these concerns are easy to get wrong, the ecosystem largely moved to dedicated libraries such as React Query (TanStack Query) or SWR, and to framework data loaders and Server Components. They provide caching, deduplication, retries, and cancellation for free. Raw `useEffect` fetching is fine for simple cases but is not the recommended default for serious apps.

```
useEffect(() => {
  const ctrl = new AbortController();
  fetch(`/api/user/${id}`, { signal: ctrl.signal })
    .then(r => r.json()).then(setUser)
    .catch(e => { if (e.name !== "AbortError") setError(e); });
  return () => ctrl.abort();
}, [id]);
```

GOTCHA

Without cancellation or an "ignore" flag, switching between two IDs fast can leave you showing the wrong user. Always guard against out-of-order responses.

37 useId

HOOKS

Q What is useId and why does it exist?

A `useId` generates a unique, stable string ID that is consistent between the server-rendered and client-rendered output. Its main purpose is accessibility: linking a form label to its input with `htmlFor` / `id`, or wiring `aria-describedby`, without hardcoding IDs that could collide when a component appears multiple times on a page.

The WHY is **server-side rendering hydration**. If you generated an ID with a counter or `Math.random()` during render, the server and client would produce different strings, and React's hydration would detect a mismatch and warn or misbehave. `useId` produces the same deterministic value on both sides because React derives it from the component's position in the tree.

It is explicitly **not** for generating keys in a list; list keys should come from your data. Nor is it a random-value generator. For multiple related IDs in one component, call it once and append suffixes to the single base value rather than calling the Hook many times.

```
function Field() {
  const id = useId();
  return (
    <>
      <label htmlFor={id}>Name</label>
      <input id={id} />
    </>
  );
}
```

GOTCHA

The generated string contains colons (like `:r0:`) and is not a valid CSS selector by default. Use it as an attribute value, not in `querySelector` without escaping.

38 useImperativeHandle

HOOKS

Q What is useImperativeHandle, and how does it work with forwardRef?

A `useImperativeHandle` customizes the value that a parent receives when it attaches a ref to your component. Normally a ref to a component exposes nothing useful (or, with `forwardRef`, a raw DOM node). This Hook lets you expose a **deliberate imperative API** instead, such as `focus()`, `scrollToTop()`, or `reset()`, hiding internal details.

It must be paired with ref forwarding. Historically you wrapped the component in `forwardRef` so the parent's ref reaches the child; inside, `useImperativeHandle(ref, () => ({ ... }), deps)` defines the object the parent sees. In React 19, `ref` can be passed as a regular prop, so `forwardRef` is often no longer needed, but the concept is identical.

The WHY, and the caution: React is declarative, so imperative handles should be rare. They are justified for genuinely imperative actions the DOM itself models, focus, text selection, media playback, animations, where expressing the action through props would be awkward. Overusing them signals that data should probably flow through props and state instead. Expose the smallest surface you can.

```
const Input = forwardRef((props, ref) => {
  const inner = useRef(null);
  useImperativeHandle(ref, () => ({
    focus: () => inner.current.focus(),
  }), []);
  return <input ref={inner} />;
});
```

GOTCHA

Reach for props/state first. If a parent needs to "tell" a child something, an imperative handle is usually a last resort, not the first tool.

39 useTransition & useDeferredValue

HOOKS

Q What do useTransition and useDeferredValue do in concurrent React?

A Both are concurrent-feature Hooks that let you mark some UI updates as lower priority, so urgent updates (like typing) stay responsive while expensive updates (like filtering a huge list) happen without blocking. They address the problem that a single heavy state update can freeze the input the user is interacting with.

`useTransition` gives you `[isPending, startTransition]`. You wrap a state update in `startTransition` to mark it **non-urgent**; React can interrupt and defer that render to keep urgent updates smooth, and `isPending` lets you show a subtle loading indicator. Use it when you control the state update and want to deprioritize the work it triggers.

`useDeferredValue` takes a value and returns a **deferred copy** that lags behind during heavy work. You use it when you receive a value (a prop or state you do not set yourself) and want to render an expensive view from a "stale but stable" version while the fresh value catches up. Roughly: `useTransition` defers an *update you trigger*; `useDeferredValue` defers a *value you consume*. Neither replaces debouncing entirely, but both reduce jank by yielding to the browser.

```
const [isPending, startTransition] = useTransition();
startTransition(() => setQuery(next)); // low priority

const deferredQuery = useDeferredValue(query);
```

GOTCHA

Only non-urgent updates belong in a transition. Never wrap a controlled input's own value update in `startTransition`, or the field will feel laggy.

40 useSyncExternalStore

HOOKS

Q What is useSyncExternalStore and when do you need it?

A `useSyncExternalStore` is the official Hook for subscribing a component to an **external store**, any mutable data source outside React such as a Redux store, a browser API, or a global event emitter. It exists so libraries can integrate with React's concurrent rendering safely, avoiding a class of bugs called "tearing" where different parts of the UI momentarily show inconsistent values from the same source.

It takes three arguments: a `subscribe` function that registers a callback and returns an unsubscribe function; a `getSnapshot` function that returns the store's current value; and an optional `getServerSnapshot` for SSR. React calls `getSnapshot` to read the value and re-renders whenever `subscribe` signals a change, guaranteeing the whole tree reads a single consistent snapshot per render.

The WHY behind it over a manual effect-plus-state pattern: with concurrent rendering, React can pause and resume work, and an ad-hoc subscription can read the store at inconsistent moments, tearing the UI. This Hook coordinates reads with React's scheduler. Most app developers never call it directly; it is primarily a building block that state-management libraries use under the hood.

```
const width = useSyncExternalStore(
  (cb) => { window.addEventListener("resize", cb);
    return () => window.removeEventListener("resize", cb); },
  () => window.innerWidth
);
```

GOTCHA

`getSnapshot` must return a cached, referentially stable value when nothing changed. Returning a fresh object each call causes an infinite render loop.

41 Prop Drilling

STATE

Q What is prop drilling, and how do you avoid it?

A Prop drilling is passing data through many intermediate components that do not use it themselves, purely to reach a deeply nested consumer. It is a symptom, not a sin: a prop passed one or two levels is fine and explicit. It becomes a problem when a value threads through five or six layers, coupling components to data they neither read nor care about, making refactors painful and signatures noisy.

The reason it hurts is that every intermediate component now depends on the prop's name and type. Rename or reshape the data and every link in the chain must change. It also obscures intent: reading a mid-level component, you cannot tell what it actually uses versus what it merely forwards.

Solutions, in order of weight: lift state only as high as needed; use component composition so children are passed as `children` or render props rather than data through props; use React Context for truly cross-cutting concerns like theme or auth; or reach for a state library when the shared state is large and frequently updated. Composition is the most underused fix.

```
// Composition sidesteps drilling: Layout never sees `user`
function App() {
  return (
    <Layout sidebar={<Profile user={user} />}>
      <Dashboard user={user} />
    </Layout>
  );
}
```

GOTCHA

Reaching for Context or Redux at the first sign of two-level passing is over-engineering. Try composition first; it keeps data flow local and testable without a global scope.

42 Context API in Depth

CONTEXT

Q How does the Context API work, from Provider to Consumer?

A Context provides a way to share a value across a subtree without passing it explicitly at each level. You create a context with `createContext(defaultValue)`, wrap part of the tree in its `Provider` with a `value` prop, and any descendant reads that value with the `useContext` hook. The default value is only used when a component consumes the context with no matching Provider above it.

The mental model is a wormhole: the Provider defines a value at one point in the tree, and consumers anywhere below tap into it directly, skipping the intermediate components entirely. When the Provider's `value` changes, every consuming component re-renders, regardless of whether an intervening component re-rendered.

The older `Context.Consumer` render-prop API still exists and is occasionally useful in class components or when you need to consume conditionally in JSX, but in modern function components `useContext` is the idiomatic choice: cleaner, and it composes with other hooks.

```
const ThemeContext = createContext('light');

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Button() {
  const theme = useContext(ThemeContext); // "dark"
  return <button className={theme} />;
}
```

GOTCHA

Consumers read the nearest Provider above them. Nesting Providers of the same context is legal and lets you override the value for a subtree, which surprises people expecting a single global value.

43 Context Performance Pitfalls

PERFORMANCE

Q Why does Context cause unnecessary re-renders, and how do you fix it?

A When a Provider's `value` changes by reference, every component that calls `useContext` for that context re-renders, even if it only uses a slice of the value that did not change. Context has no selector mechanism: it is all-or-nothing subscription. This is the root of most Context performance complaints.

The most common self-inflicted wound is passing a fresh object literal as `value` on every render. Because `{ user, setUser }` is a new reference each time, consumers re-render on every parent render even when nothing meaningful changed. Wrap the value in `useMemo` so its identity is stable between renders.

For genuinely distinct concerns, split one big context into several smaller ones so a change to theme does not re-render auth consumers. If you need selector-style subscriptions where a consumer re-renders only when its slice changes, that is where libraries like Redux (with `useSelector`) or Zustand shine, or the `use-context-selector` library.

```
const value = useMemo(
  () => ({ user, setUser }),
  [user]
);
return <UserContext.Provider value={value}>{children}</UserContext.Provider>;
```

GOTCHA

`React.memo` on a consumer does **not** stop re-renders triggered by a context value change. memo only guards against parent prop changes; context updates bypass it entirely.

44 Context vs State Library

ARCHITECTURE

Q When should you use Context versus a dedicated state management library?

A Context is a dependency-injection tool, not a state manager. It excels at distributing values that are read often but change rarely: theme, locale, the current user, a configured client. When the shared data changes frequently or is read by many components that each care about different slices, Context's lack of selective subscription becomes a performance liability, and that is the signal to reach for a library.

A state library like Redux, Zustand, or Jotai adds selectors (subscribe to a slice, re-render only on relevant change), devtools, middleware, and a structured update model. These matter when state is complex, updated from many places, or benefits from time-travel debugging and predictable transitions.

A pragmatic rule: start with local `useState`, promote to Context for low-frequency global values, and adopt a library when you feel Context re-render pain or need async orchestration, caching, or a formal action log. Do not skip straight to Redux for a theme toggle.

GOTCHA

Context plus `useReducer` is often pitched as a Redux replacement, but it still lacks selective subscription. It works for small apps yet re-renders all consumers on every dispatch, so it does not scale the way people assume.

45 Redux Core Concepts

REDUX

Q Explain Redux's core concepts and its one-way data flow.

A Redux centralizes application state in a single immutable object called the store. You never mutate it directly. Instead you dispatch an action, a plain object describing what happened (with a `type` and optional `payload`), and a reducer, a pure function `(state, action) => newState`, computes the next state. The store then notifies subscribers, and the UI re-renders.

The value is predictability. Because reducers are pure and state changes only through dispatched actions, every state transition is explicit, replayable, and inspectable. This is what enables time-travel debugging and makes complex state flows auditable: you can log every action and reconstruct exactly how the app reached its current state.

The one-way flow is the discipline that makes it work: UI dispatches action, reducer produces new state, store updates, UI reflects new state. Data never flows backward or sideways. This unidirectional cycle is what keeps large applications from becoming a tangle of two-way bindings.

```
function counterReducer(state = { count: 0 }, action) {
  switch (action.type) {
    case 'INCREMENT':
      return { count: state.count + 1 };
    default:
      return state;
  }
}
```

GOTCHA

Reducers must be pure: no API calls, no mutation, no `Date.now()` or random values. Side effects and impurity break time-travel and make the same action produce different results, defeating Redux's core guarantee.

46 Redux Toolkit

REDUX

Q Why did Redux Toolkit replace classic Redux, and what is createSlice?

A Classic Redux was correct but ceremonial: hand-written action-type constants, action creators, switch-statement reducers, and manual immutable updates with spread operators. This boilerplate discouraged people and invited bugs. Redux Toolkit (RTK) is now the official, recommended way to write Redux, collapsing that ceremony into a fraction of the code.

The centerpiece is `createSlice`, which generates action creators and action types automatically from the reducer names you define. Crucially, it uses Immer under the hood, so you write what looks like mutating code inside reducers and Immer produces the correct immutable update behind the scenes. That single feature eliminates the most error-prone part of classic Redux.

RTK also bundles `configureStore` (with sensible defaults, devtools, and thunk middleware preinstalled) and `createAsyncThunk` for async flows. The result is less code, fewer mistakes, and no loss of Redux's predictability guarantees.

```
const counterSlice = createSlice({
  name: 'counter',
  initialState: { count: 0 },
  reducers: {
    increment: (state) => { state.count += 1; }, // Immer makes this safe
  },
});
export const { increment } = counterSlice.actions;
```

GOTCHA

The "mutating" syntax only works inside `createSlice` / `createReducer` where Immer wraps it. Mutate state anywhere else, or return a value **and** mutate in the same reducer, and you get broken updates.

47 useSelector and useDispatch

REDUX

Q How do useSelector and useDispatch connect components to the store?

A These are the React-Redux hooks that replaced the older `connect` higher-order component. `useSelector` takes a selector function that extracts a slice of state and subscribes the component to just that slice. `useDispatch` returns the store's dispatch function so the component can send actions.

The key behavior of `useSelector` is that it re-renders the component only when the selected value changes, compared with a strict-equality (`===`) check by default. This selective subscription is what Context lacks and is a major reason Redux scales. If your selector returns a new object or array literal each call, the reference always differs and you re-render every time, so return primitives or memoize.

`useDispatch` returns a stable reference, so it is safe to omit from or include in dependency arrays without concern. Together they give a component a precise read/write channel to global state.

```
const count = useSelector((s) => s.counter.count);
const dispatch = useDispatch();

return <button onClick={() => dispatch(increment())}>{count}</button>;
```

GOTCHA

Returning `{ a, b }` from a selector re-renders on every dispatch because the object is new each time. Use separate `useSelector` calls, or pass `shallowEqual` as the second argument, or use RTK's `createSelector`.

48 Redux Middleware and Thunks

ASYNC

Q What is Redux middleware, and how do thunks handle async logic?

A Reducers must be pure, so async work like API calls cannot live inside them. Middleware sits between dispatching an action and the moment it reaches the reducer, intercepting each action to log it, transform it, delay it, or perform side effects. It is the official extension point for anything impure.

A thunk is the standard async pattern, enabled by `redux-thunk` (bundled by default in RTK). Instead of dispatching a plain object, you dispatch a function. The thunk middleware detects that the action is a function and calls it with `dispatch` and `getState`, letting you run async logic and dispatch real actions when it resolves, typically pending/fulfilled/rejected phases.

RTK's `createAsyncThunk` formalizes this: you give it a type prefix and an async function, and it auto-dispatches `pending`, `fulfilled`, and `rejected` actions you handle in `extraReducers`. That covers loading and error states without hand-rolled bookkeeping.

```
const fetchUser = createAsyncThunk(
  'user/fetch',
  async (id) => {
    const res = await fetch(`/api/users/${id}`);
    return res.json();
  }
);
```

GOTCHA

Thunks work but are verbose for server data. For fetching, caching, and re-fetching, RTK Query or React Query is a better fit than managing loading/error/cache state by hand in Redux.

49 Choosing: Redux vs Context vs Local

ARCHITECTURE

Q How do you choose between Redux, Context, and local state?

A Match the tool to the state's scope and change frequency. Local `useState` is the default for anything a single component or a small subtree owns: form inputs, toggles, hover state. Keeping state local is cheapest, most encapsulated, and easiest to reason about, so lift it out only when you must.

Context fits low-frequency global values read widely but written rarely, such as theme, language, or the authenticated user. It distributes without drilling but re-renders all consumers on change, so it is a poor fit for hot, frequently updated data.

A library like Redux earns its keep when state is genuinely global, updated from many places, complex enough to benefit from a formal action log and devtools, or shared across distant parts of the tree with selective subscription. The honest modern caveat: much of what teams once put in Redux is really server state, better handled by React Query, shrinking Redux's remaining role to true client state.

GOTCHA

Do not globalize state out of habit. Every value in a global store is a value any component can change from anywhere, which erodes the locality that makes bugs easy to trace. Prefer the smallest scope that works.

50 Zustand and Jotai

LIBRARIES

Q What are Zustand and Jotai, and how do they differ from Redux?

A Both are lightweight state libraries that trade Redux's ceremony for a minimal API while keeping selective subscription. They exist because Redux, even with Toolkit, can feel heavy for apps that just need a shared store without actions, reducers, and boilerplate.

Zustand uses a single store defined by a hook. You call `create` with a function that returns state and updater functions, then read slices with a selector inside the hook. Components subscribe only to the slices they select, so re-renders stay narrow, and there is no Provider to wrap the tree. It follows a store-centric model much like Redux but with a fraction of the code.

Jotai takes an atomic, bottom-up approach inspired by Recoil: state is composed of small independent `atom`s, and components subscribe to individual atoms with `useAtom`. Derived atoms compute from other atoms. It suits state that is naturally granular and interdependent, where a single big store would feel awkward.

```
// Zustand: store as a hook, no Provider needed
const useStore = create((set) => ({
  count: 0,
  inc: () => set((s) => ({ count: s.count + 1 })),
}));

const count = useStore((s) => s.count);
```

GOTCHA

These solve client state, not server state. Pairing Zustand or Jotai with React Query (server data) rather than stuffing fetched data into the store is the pattern that keeps both concerns clean.

51 Client State vs Server State

DATA

Q What is the distinction between client state and server state?

A This distinction reshaped how modern React apps manage data. Client state is owned entirely by the browser: form inputs, modal open/closed, selected tab, theme. It is synchronous, always up to date, and only your app touches it. Server state is data that lives on a remote server, which your app merely borrows a cached copy of: user records, product lists, anything fetched over the network.

The reason this matters is that server state has properties client state never has. It is asynchronous, it can become stale the instant you fetch it because someone else may change it, it is shared across clients, and it needs caching, background re-fetching, and de-duplication. Treating it like client state, by dumping fetched data into Redux or `useState`, forces you to hand-build all of that machinery.

Recognizing that most of your "global state" is actually cached server state is the insight behind React Query, SWR, and RTK Query. Once separated, client state shrinks to a small amount of genuinely local UI concerns, and the fetching, caching, and syncing is delegated to a purpose-built tool.

GOTCHA

The classic anti-pattern is copying server data into local state with `useEffect` plus `useState`. You then own staleness, refetch, and cache invalidation manually, reinventing a fragile version of what a data library gives you for free.

52 React Query / TanStack Query

DATA

Q What problem does React Query (TanStack Query) solve?

A React Query manages server state: fetching, caching, synchronizing, and updating remote data. The problem it solves is that the naive approach, `useEffect` to fetch plus `useState` for data, loading, and error, is repetitive and quietly wrong. It ignores caching, refetching, request de-duplication, and background updates, so every screen reinvents them inconsistently.

You wrap a request in `useQuery` with a query key and a fetch function, and it returns `data`, `isLoading`, `error`, and more, while handling caching by key, de-duplicating concurrent requests for the same key, refetching stale data in the background, and retrying failures. Mutations go through `useMutation`.

The result is dramatically less code and a better UX: cached data shows instantly while fresh data loads behind the scenes, multiple components requesting the same key share one request, and the cache stays coherent. It replaces a large amount of what teams previously built in Redux for data fetching.

```
const { data, isLoading, error } = useQuery({
  queryKey: ['user', id],
  queryFn: () => fetch(`/api/users/${id}`).then((r) => r.json()),
});
```

GOTCHA

The query key is the cache identity. Forgetting to include a variable like `id` in the key means different requests share one cache entry, so you see stale or wrong data when the variable changes.

53 Caching, Staleness, Invalidation

DATA

Q How do caching, staleness, and invalidation work in React Query?

A React Query caches each query's result under its key and governs freshness with two timers. `staleTime` is how long data is considered fresh; while fresh, it is served from cache with no refetch. Once it passes `staleTime` it becomes stale and eligible for background refetching on triggers like window refocus, remount, or reconnect. The default `staleTime` is 0, so data is stale immediately unless you raise it.

Separately, `gcTime` (formerly `cacheTime`) controls how long an unused query's data lingers in memory after no component is using it, before garbage collection. Staleness governs refetching; `gcTime` governs eviction. Conflating the two is a common source of confusion.

Invalidation is the explicit push mechanism: after a mutation you call `queryClient.invalidateQueries` with the affected key, marking those queries stale and triggering a refetch so the UI reflects the server's new truth. This keeps the cache coherent after writes without manually patching cached values.

```
const qc = useQueryClient();
useMutation({
  mutationFn: updateUser,
  onSuccess: () => qc.invalidateQueries({ queryKey: ['user'] }),
});
```

GOTCHA

Leaving `staleTime` at 0 can cause surprisingly frequent refetches on every focus and remount. Tune it to your data's real volatility; user profiles tolerate minutes, a live price feed does not.

54 Optimistic Updates

DATA

Q What are optimistic updates, and how do you implement them safely?

A An optimistic update applies a change to the UI immediately, before the server confirms it, assuming the request will succeed. It makes an app feel instant: a liked post fills in the moment you click rather than after a network round trip. The tradeoff is that you must handle the case where the server actually rejects the change.

The safe pattern has three phases. Before mutating, cancel in-flight refetches and snapshot the current cache so you can restore it. Then optimistically write the expected new value into the cache. If the mutation fails, roll back to the snapshot; on settle, invalidate the query to resync with the server's authoritative state. React Query exposes exactly these hooks: `onMutate`, `onError`, and `onSettled`.

The reason rollback is non-negotiable is that without it a failed request leaves the UI showing a change that never persisted, silently lying to the user. Optimism must always be paired with a correction path.

```
useMutation({
  mutationFn: toggleLike,
  onMutate: async (next) => {
    await qc.cancelQueries({ queryKey: ['post'] });
    const prev = qc.getQueryData(['post']);
    qc.setQueryData(['post'], next);
    return { prev }; // context for rollback
  },
  onError: (e, v, ctx) => qc.setQueryData(['post'], ctx.prev),
});
```

GOTCHA

Skipping the pre-mutation `cancelQueries` lets an in-flight background refetch resolve after your optimistic write and overwrite it with stale server data, undoing the update mid-flight.

55 Immutability in React State

STATE

Q Why must React state be immutable, and how do you update nested state?

A React decides whether to re-render by comparing the previous and next state references with `Object.is`, a shallow check. If you mutate an object or array in place, its reference does not change, so React sees no difference and skips the render, even though the contents changed. Immutability, creating a new object/array for every change, is what makes change detection work.

To update, you produce a copy with the change applied rather than editing the original. For flat state this is a spread. For nested state you must copy every level along the path to the changed value, because a shallow copy of the outer object still shares references to inner objects, and mutating those inner objects is still a mutation.

Deep nesting makes manual spreading verbose and bug-prone, which is why Immer exists: you write mutating-looking code against a draft and Immer produces the correct immutable copy. Redux Toolkit builds this in. A flatter state shape also reduces the depth you have to spread through.

```
// Update nested value immutably
setUser((u) => ({
  ...u,
  address: { ...u.address, city: 'Berlin' },
}));

// Arrays: map/filter/concat, never push/splice on state
setItems((items) => items.map((i) =>
  i.id === id ? { ...i, done: true } : i
));
```

GOTCHA

`const copy = { ...state }` is only a shallow copy. `copy.nested.x = 1` mutates the original's nested object because both share the same reference, a silent bug that skips renders.

56 Normalizing State Shape

STATE

Q What does it mean to normalize state, and why do it?

A Normalizing state means storing collections as a lookup table keyed by id plus an array of ids for ordering, rather than as nested arrays of full objects. It borrows the idea from database design: each entity lives in exactly one place, referenced elsewhere by id. This avoids duplication and the inconsistencies duplication breeds.

The problem it solves appears when the same entity is nested in multiple places, say a user embedded in several posts. Update that user and you must find and patch every copy, and any you miss goes stale. With a normalized `byId` map, the entity exists once, so an update touches a single location and every reference sees it.

Normalization also makes lookups $O(1)$ by id instead of scanning an array, and simplifies immutable updates because you change one keyed entry rather than mapping over a nested tree. Redux Toolkit's `createEntityAdapter` provides this shape and its reducers out of the box.

```
// Normalized shape
{
  byId: {
    'a1': { id: 'a1', title: 'Hello' },
    'a2': { id: 'a2', title: 'World' },
  },
  allIds: ['a1', 'a2'],
}
```

GOTCHA

Do not normalize everything reflexively. For small, self-contained, non-shared data, nested arrays are simpler and clearer. Normalization pays off mainly with relational, shared, or frequently updated entities.

57 Derived State

STATE

Q What is derived state, and how do you avoid redundant state?

A Derived state is any value you can compute from existing state or props rather than storing separately. The guiding principle is that state should hold the minimal source of truth; anything calculable from it should be calculated during render, not duplicated into its own state variable. Redundant state is the root of a whole class of sync bugs.

The failure mode is storing a full name alongside first and last, or a filtered list alongside the raw list plus filter. Now two things must be kept in step, and the moment one updates without the other, they diverge and the UI shows contradictory data.

Deriving on render makes divergence impossible because there is only one source.

Just compute the value inline during render. If the computation is genuinely expensive and the inputs change rarely, wrap it in `useMemo` as a performance optimization, but that is caching a derivation, not storing state. Reserve `useState` for values that cannot be derived.

```
// Redundant: fullName can drift from first/last
const [fullName, setFullName] = useState(''); // avoid

// Derived: single source of truth
const fullName = `${first} ${last}`;
const visible = items.filter(i => i.active);
```

GOTCHA

Syncing props into state with `useEffect` to "derive" a value is an anti-pattern: it renders once with stale data, then again after the effect. Compute during render instead of mirroring props into state.

58 Forms at Scale

FORMS

Q React Hook Form versus controlled state: which and why?

A Fully controlled forms bind every input's value to state and update it on every keystroke via `onChange`. This is fine for a few fields, but at scale each keystroke re-renders the whole form, and you hand-manage validation, errors, and touched state. The re-render cost and boilerplate grow with field count.

React Hook Form takes an uncontrolled approach: it registers inputs with refs and reads their values directly from the DOM, so typing does not trigger React re-renders for the whole form. This keeps large forms fast and cuts boilerplate, since validation, error tracking, and submission are handled by the library. You subscribe to only the pieces you need.

Choose controlled state when you need React to react to every keystroke, for example live cross-field computation or tightly coupled UI. Choose React Hook Form (or similar) for larger forms where performance and reduced boilerplate matter, which is most real-world forms. It also integrates schema validation via resolvers like Zod or Yup.

```
const { register, handleSubmit, formState: { errors } } = useForm();

return (
  <form onSubmit={handleSubmit(onValid)}>
    <input {...register('email', { required: true })} />
    {errors.email && <span>Required</span>}
  </form>
);
```

GOTCHA

Mixing paradigms trips people up: passing `value` without `onChange` makes an input read-only and logs a warning. With React Hook Form, use `register` (uncontrolled) or the `Controller` wrapper for controlled components, not raw `value`.

59 Debouncing and Throttling

PERFORMANCE

Q How do you debounce and throttle input in React, and what is the difference?

A Both limit how often a rapidly-firing handler actually does expensive work, but differently. Debouncing waits until activity stops: it fires only after a quiet gap of N milliseconds, so a search box triggers one request after the user pauses typing, not on every keystroke. Throttling instead fires at most once per interval regardless of how many events arrive, which suits continuous streams like scroll or resize.

In React the subtlety is that a naive debounced function recreated on every render loses its internal timer, so it never actually debounces. You must keep the debounced function stable across renders with `useMemo` / `useRef`, or debounce a value with a `useEffect` plus `setTimeout` and cleanup.

The value-debounce pattern is the most idiomatic: hold the raw input in state, and in an effect start a timer to copy it into a debounced value, clearing the timer on each change so only the final value after the pause survives. Downstream effects then depend on the debounced value.

```
function useDebounced(value, delay = 300) {
  const [v, setV] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setV(value), delay);
    return () => clearTimeout(id); // reset on each change
  }, [value, delay]);
  return v;
}
```

GOTCHA

Defining `debounce(fn)` inline in the component body creates a brand-new debounced function every render, so its timer never persists and nothing is actually debounced. Stabilize it with a ref or memo.

60 Error Boundaries

RESILIENCE

Q What do error boundaries catch, and what do they not catch?

A An error boundary is a component that catches JavaScript errors thrown during rendering, in lifecycle methods, and in constructors of the components below it, then renders a fallback UI instead of letting the whole React tree unmount. Without one, an uncaught render error crashes the entire app to a blank screen, so boundaries contain failures to a section.

They are still class components: you implement `static getDerivedStateFromError` to render the fallback and `componentDidCatch` to log the error. There is no hook equivalent yet, so people typically use a small class boundary or the `react-error-boundary` library, which wraps this in a friendlier API.

Crucially, boundaries do not catch errors in event handlers, in asynchronous code like `setTimeout` or promise callbacks, during server-side rendering, or thrown by the boundary itself. Those fall outside React's render pipeline, so you handle them with ordinary `try/catch`. That gap surprises people who expect a boundary to catch everything.

```
class ErrorBoundary extends React.Component {
  state = { hasError: false };
  static getDerivedStateFromError() {
    return { hasError: true };
  }
  componentDidCatch(error, info) { logError(error, info); }
  render() {
    return this.state.hasError
      ? <Fallback />
      : this.props.children;
  }
}
```

GOTCHA

Event-handler errors are the big blind spot. A click handler that throws is not caught by any boundary; wrap it in `try/catch` and set error state yourself, or route it through `react-error-boundary`'s reset API.

61 Reconciliation & Diffing

RENDERING

Q What is reconciliation, and how does React's diffing algorithm work?

A Reconciliation is the process React uses to figure out what changed between two renders. When a component renders, React produces a tree of React elements (the virtual representation). On the next render it produces a new tree, then **diffs** the new tree against the previous one to compute the minimal set of DOM mutations needed. This lets you write declarative "describe the whole UI" code while React handles imperative DOM updates efficiently.

A general tree-diff is $O(n^3)$, which is far too slow for UIs. React uses a heuristic $O(n)$ algorithm built on two assumptions: (1) two elements of different **type** produce different trees, and (2) developers can hint at stable identity across renders using **key**. Because of assumption (1), if a `<div>` becomes a `<span>`, React tears down the entire subtree and rebuilds it rather than trying to match children.

When the type is the same, React keeps the underlying DOM node and only updates changed attributes, then recurses into children. For children of the same type it walks both lists in order; without keys, position *is* the identity, so inserting an item at the front can cause every subsequent item to be treated as "changed." Keys fix this by giving elements stable identities independent of position.

```
function App({ swap }) {  
  // Different type => full remount of the subtree  
  return swap  
    ? <section><Child /></section>  
    : <div><Child /></div>;  
}
```

GOTCHA

"Virtual DOM" is not inherently faster than direct DOM manipulation. Its value is the *programming model*: you describe UI as a function of state, and React batches and minimizes real DOM writes. Hand-tuned imperative code can be faster; React trades a little speed for a lot of maintainability.

62 Keys & Reconciliation

RENDERING

Q How do keys affect reconciliation and re-renders?

A A **key** tells React which element in a list corresponds to which element in the previous render, independent of order. During reconciliation of a list, React builds a map from key to previous element and matches by key rather than by index. This means React can move, insert, or remove list items surgically instead of mutating every node from the change point onward.

Using the array **index** as a key is the classic anti-pattern. If the list is reordered, filtered, or has items inserted at the front, index keys make React think item content changed in place, so component state (uncontrolled inputs, focus, animations) gets associated with the wrong item. It also defeats the optimization: React re-renders and re-commits elements that only moved.

Changing a component's key is also a deliberate tool: because a different key means a different identity, React unmounts the old instance and mounts a fresh one, resetting all internal state. This is the idiomatic way to "reset" a component (for example, a form) when a prop like a record ID changes.

```
// Reset internal state by changing key
<ProfileForm key={userId} user={user} />
```

GOTCHA

Keys only need to be unique among siblings, not globally. And keys are a hint to React, never passed to the component as a prop; if you need the value inside, pass it separately (e.g. **id={userId}**).

63 What Triggers a Re-render

RENDERING

Q What actually causes a component to re-render?

A There are exactly three things that cause a function component to re-render: (1) its own state changes via a `useState` / `useReducer` setter, (2) a subscribed context value it consumes changes, or (3) its parent re-renders. That third one surprises people: by default, when a component re-renders, React re-renders *all* of its children, regardless of whether their props changed.

Notably, "props changed" is *not* on that list. Props changing don't independently trigger a re-render; a child re-renders because its parent re-rendered (which is what produced the new props). This distinction matters because it explains why `React.memo` works: memo lets a child opt out of the "parent re-rendered" rule by comparing props and skipping the render when they're shallow-equal.

Re-rendering is not the same as updating the DOM. A re-render runs your component function and reconciles; if the output is equivalent, React commits zero DOM changes. Re-renders are usually cheap, but they add up in large trees, and they run all your hooks and inline calculations each time, which is where wasted work accumulates.

```
function Parent() {  
  const [n, setN] = useState(0);  
  // Child re-renders on every click, even though  
  // its props never change – because Parent did.  
  return <><button onClick={() => setN(n + 1)} /><Child /></>;  
}
```

GOTCHA

Calling a setter with a value equal to the current state (via `Object.is`) causes React to bail out — but it may still render the component once before bailing. Don't rely on `setState`-with-same-value as a performance guarantee.

64 React.memo

PERFORMANCE

Q What does React.memo do, and what are its limits?

A `React.memo` wraps a component so that React skips re-rendering it when its parent re-renders, as long as its props are shallow-equal to the previous props. It memoizes the rendered output keyed on props. This breaks the default "parent renders → child renders" chain, which is valuable when a cheap parent re-renders often but an expensive child's props rarely change.

Its comparison is shallow by default: it compares each prop with `Object.is`. This is exactly why passing a fresh object, array, or function literal as a prop silently defeats memo — a new reference each render is never equal to the last. You can pass a custom comparator as the second argument, but deep comparison is often more expensive than the render you're trying to avoid.

memo also does *not* stop re-renders caused by the component's own state or by a consumed context. It only guards the "parent re-rendered" path. Wrapping everything in memo is a common mistake: the comparison itself costs time and memory, and most renders are cheap enough that the bookkeeping is net-negative.

```
const Row = React.memo(function Row({ label, onSelect }) {  
  return <li onClick={onSelect}>{label}</li>;  
});  
// Only helps if `onSelect` is a stable reference  
// (e.g. wrapped in useCallback).
```

GOTCHA

`children` is a prop too. If you pass JSX children, they're a new element object every render, so a memoized wrapper around `{children}` won't skip renders unless the parent also memoizes those children.

65 useMemo vs useCallback

PERFORMANCE

Q When do useMemo and useCallback actually help performance?

A `useMemo` caches the *result* of a computation between renders, recomputing only when its dependencies change. `useCallback` caches a *function reference* between renders. In fact `useCallback(fn, deps)` is just `useMemo(() => fn, deps)`. Both exist to preserve referential stability or to avoid repeating expensive work.

They help in two concrete situations. First, when a value is an expensive computation (sorting a large list, parsing) and you want to skip redoing it on unrelated re-renders — that's `useMemo` for CPU. Second, when a value or function is passed as a prop to a `React.memo` child or used in another hook's dependency array, and you need a stable reference so the downstream optimization actually fires — that's referential stability, served by either hook.

If neither condition holds, they don't help and add overhead: React still stores the deps, compares them each render, and keeps the cached value in memory. Memoizing a trivial calculation or a callback passed to a non-memoized child is pure cost. Measure before reaching for them.

```
const sorted = useMemo(
  () => items.slice().sort(compare),
  [items]
); // recompute only when items changes

const handleAdd = useCallback(
  (id) => setItems((xs) => [...xs, id]),
  []
); // stable ref for a memoized child
```

GOTCHA

`useMemo` is a performance hint, not a semantic guarantee. React may discard the cache (e.g. for memory) and recompute. Never put logic with side effects or correctness-critical identity inside it — use `useRef` or state for values that must persist.

66 Inline Props & Memoization

PERFORMANCE

Q Why does passing inline objects/functions as props hurt memoization?

A Every time a component renders, any object literal, array literal, or arrow function written in its JSX is created anew. Even if the contents are identical, the *reference* is different. Since `React.memo` and hook dependency arrays compare with `Object.is`, a fresh reference always looks "changed," so the memoization you thought you had never engages.

Concretely, `style={{ margin: 8 }}` and `onClick={() => doThing()}` produce brand-new values on every render. A memoized child receiving them re-renders every time its parent does. Worse, if that reference is in a `useEffect` dependency array, the effect re-runs each render, potentially causing loops or redundant network calls.

The fixes: hoist truly static objects out of the component (module scope), wrap computed objects in `useMemo`, and wrap callbacks in `useCallback`. But apply this surgically — only where a downstream memoized boundary or dependency array actually benefits. Blanket-wrapping every inline value trades one small cost for another.

```
// New object & function every render:
<Child style={{ pad: 4 }} onTap={() => go()} />

// Stable references:
const style = useMemo(() => ({ pad: 4 }), []);
const onTap = useCallback(() => go(), []);
<Child style={style} onTap={onTap} />
```

GOTCHA

If the child is *not* wrapped in `React.memo` and the value isn't a dependency anywhere, stabilizing the reference does nothing for performance — the child re-renders because its parent did, regardless of prop identity.

67 Diagnosing Wasted Renders

PERFORMANCE

Q How do you diagnose and fix unnecessary re-renders?

A Start by confirming there's a real problem: unnecessary re-renders only matter when they're frequent and expensive. Use the React DevTools Profiler (or the "Highlight updates when components render" option) to see which components render and why. The Profiler tells you, per commit, which components rendered and attributes the render to props, state, hooks, or parent.

Common causes and fixes: a high-frequency state update (like mouse position or input value) living too high in the tree — push that state down into a smaller subtree so fewer components re-render. Unstable props defeating `React.memo` — stabilize with `useMemo` / `useCallback`. A context value that's a fresh object each render — memoize the provider's value. An expensive child under a chatty parent — wrap the child in `React.memo`, or lift the child out via the `children-as-prop` pattern so it isn't recreated.

The children-as-prop trick is underused: if you pass an expensive component as `children` to a component that manages the frequently-changing state, the expensive child is created by the outer component and won't re-render when the inner state changes.

```
function Toggle({ children }) {  
  const [open, setOpen] = useState(false);  
  // children is created by the parent, so it  
  // does NOT re-render when `open` changes.  
  return <div>{open && children}</div>;  
}
```

GOTCHA

Don't optimize blind. Wrapping everything in memo/useMemo can make an app slower and harder to read. Profile first, fix the hot path, then re-measure to confirm the change actually helped.

68 Code Splitting

PERFORMANCE

Q How do `React.lazy` and `Suspense` enable code splitting?

A Code splitting breaks your bundle into chunks that load on demand instead of shipping everything up front. `React.lazy` takes a function that returns a dynamic `import()` and produces a component that isn't loaded until it first renders. The bundler (Webpack, Vite, etc.) sees the dynamic import and automatically emits a separate chunk for that module.

Because the code loads asynchronously, React needs something to show while it's fetching. That's what `Suspense` provides: any lazy component must render beneath a `<Suspense fallback={...}>` boundary, which displays the fallback until the chunk resolves. One boundary can cover many lazy components; the fallback shows until all suspended children are ready.

Good places to split: route-level components (each page becomes a chunk), heavy modals or editors, and rarely-used feature panels. The trade-off is an extra network round trip when the component first appears, so avoid over-splitting tiny components where the request overhead exceeds the savings. Preloading (calling the import on hover/intent) can hide that latency.

```
const Editor = React.lazy(() => import('./Editor'));

function Page() {
  return (
    <Suspense fallback={<Spinner />}>
      <Editor />
    </Suspense>
  );
}
```

GOTCHA

`React.lazy` only supports default exports. For a named export, re-map it:

`import('./m').then(m => ({ default: m.Named })))`. Also wrap lazy boundaries in an error boundary — a failed chunk fetch (e.g. after a deploy) throws.

69 List Virtualization

PERFORMANCE

Q What is list virtualization (windowing) and when do you need it?

A Virtualization renders only the list items currently visible in the viewport (plus a small buffer), instead of mounting all of them. If you have 10,000 rows but only 20 fit on screen, you mount ~25 DOM nodes and swap their contents as the user scrolls. This keeps DOM size, memory, and render/commit time bounded regardless of dataset length.

It works by measuring the scroll container, computing which items fall in the visible window from scroll offset and item height, and absolutely positioning those items inside a spacer element sized to the full list height (so the scrollbar behaves correctly).

Libraries like `react-window`, `react-virtuoso`, and TanStack Virtual handle the math, including variable-height items and dynamic measurement.

Reach for it when a list is long enough that mounting everything causes jank — typically hundreds to thousands of rows, especially with rich item content. For short lists it's unnecessary complexity. The main costs are added complexity, trickier accessibility (screen readers and in-page find won't see off-screen rows), and challenges with variable heights.

```
import { FixedSizeList } from 'react-window';

<FixedSizeList height={400} itemCount={items.length}
  itemSize={35} width={300}>
  {({ index, style }) => (
    <div style={style}>{items[index].name}</div>
  )}
</FixedSizeList>
```

GOTCHA

The per-row `style` prop from the library carries absolute positioning — you must spread it onto your row's outermost element, or rows will stack at the top. Also, Ctrl/Cmd+F won't find un-rendered rows; provide a search feature if discoverability matters.

70 DevTools Profiler

TOOLING

Q How do you use the React DevTools Profiler?

A The Profiler tab in React DevTools records a session of commits and shows, for each commit, which components rendered and how long they took. You click record, interact with the app, stop, and then inspect the flamegraph or ranked chart. It's the primary evidence-based tool for finding performance problems instead of guessing.

The **flamegraph** shows the component tree for a commit, with bar width proportional to render time; greyed-out components didn't render that commit. The **ranked** view lists components sorted by render duration. Selecting a component shows "why did this render?" (props changed, state changed, hooks changed, or parent rendered) — enable that in the Profiler settings to trace root causes quickly.

You can also step through commits with the timeline to see how many commits an interaction caused (often revealing redundant renders). For programmatic measurement, the `<Profiler>` component takes an `onRender` callback reporting actual vs. base render duration, useful for logging in environments where DevTools isn't available.

```
<Profiler id="List" onRender={(id, phase, actual) => {  
  console.log(id, phase, actual);  
}}>  
  <List />  
</Profiler>
```

GOTCHA

Profile a production-like build; the development build is intentionally slower and includes warnings/checks. Use React's profiling build so measurements reflect what users experience, not dev-mode overhead.

71 Automatic Batching

RENDERING

Q What is automatic batching in React 18?

A Batching means React groups multiple state updates into a single re-render for performance. If you call three setters, React doesn't render three times — it collects them and renders once with the final state. Before React 18, batching only happened inside React event handlers. Updates in promises, `setTimeout`, native event handlers, or after `await` were *not* batched and each triggered its own render.

React 18 introduced **automatic batching**: with `createRoot`, state updates are batched everywhere, regardless of where they originate — timeouts, promises, async functions, native events. This means fewer renders and more consistent behavior. It's enabled automatically when you opt into the concurrent root.

Batching is based on the JavaScript execution context, not on which component the state belongs to; multiple updates across different components in the same tick still batch into one render pass. If you genuinely need to force a synchronous, flushed update between two setters (rare — e.g. to measure the DOM in between), `flushSync` from `react-dom` opts out for that update.

```
function handleClick() {
  // React 18: these batch into ONE render,
  // even inside this async callback.
  fetchData().then(() => {
    setLoading(false);
    setData(result);
  });
}
```

GOTCHA

State updates are asynchronous within a batch: reading state right after calling a setter gives the *old* value. Use the functional updater form `setN(n => n + 1)` when the next value depends on the previous one, or successive updates in a batch will clobber each other.

72 Concurrent Rendering

RENDERING

Q What does concurrent rendering mean in React 18?

A Concurrent rendering is a new capability where React can prepare (render) a UI update in the background without blocking the main thread, and can pause, resume, or abandon that work. Before this, rendering was synchronous and uninterruptible: once React started rendering a tree, it ran to completion, potentially blocking user input for tens of milliseconds on large updates.

Concurrency lets React treat some updates as **low priority** and interruptible. If a more urgent update (like a keystroke) arrives while React is rendering a big, non-urgent update, it can pause the big one, handle the urgent one, then resume. Crucially, half-finished concurrent renders are never shown to the user — the results are committed atomically only when complete.

You don't get concurrency just by upgrading; it's opt-in per update through features built on it: `useTransition`, `useDeferredValue`, and streaming SSR with Suspense. What you *do* get for free by switching to `createRoot` is the foundation (and automatic batching). The mental shift is that a render is no longer a guaranteed atomic, synchronous event.

```
import { createRoot } from 'react-dom/client';

// Opts into the concurrent-capable root
const root = createRoot(document.getElementById('root'));
root.render(<App />);
```

GOTCHA

Because a component may render and then have that render thrown away, render functions must be pure — no side effects during render. Strict Mode double-invokes renders in development specifically to surface impurity that concurrency would otherwise expose in production.

73 useTransition

PERFORMANCE

Q How does useTransition keep the UI responsive?

A `useTransition` lets you mark a state update as a non-urgent **transition**, so React can keep the app responsive to more urgent updates (like typing) while the expensive update renders in the background. It returns `[isPending, startTransition]`; updates wrapped in `startTransition` are rendered at low priority and can be interrupted.

The classic case is a search box that filters a huge list. The input value should update instantly (urgent), but re-rendering the giant filtered list is expensive. By keeping the input state as a normal update and wrapping the list's derived state in a transition, keystrokes stay snappy; the list "catches up" without freezing the field. `isPending` lets you show a subtle loading indicator during the transition.

Transitions don't make the heavy render faster — they make it non-blocking and interruptible, so the urgent work isn't starved. If the user types again before the list finishes, React abandons the in-progress list render and starts fresh with the newest input. For a simpler read-only version, `useDeferredValue` defers a value without needing to wrap the setter.

```
const [isPending, startTransition] = useTransition();

function onChange(e) {
  setQuery(e.target.value);           // urgent
  startTransition(() => {
    setResults(filter(e.target.value)); // deferred
  });
}
```

GOTCHA

Only wrap state updates in `startTransition`, not the input value itself — if you defer the controlled input's own state, the field feels laggy. Also, transitions can't wrap updates driven by controlled text inputs' immediate value; keep those urgent.

74 Context Re-render Optimization

PERFORMANCE

Q How do you optimize Context to avoid excessive re-renders?

A Every component that consumes a context re-renders whenever the context's *value* changes (by `Object.is`). The most common mistake is passing a fresh object literal as the provider value: `value={{ user, setUser }}` creates a new object every render, so all consumers re-render even when nothing meaningful changed. Wrap the value in `useMemo` so it only changes when its parts do.

Second, split contexts by change frequency. If you cram rarely-changing data (theme) and frequently-changing data (mouse position) into one context, every mouse move re-renders theme consumers. Separate them into distinct providers so a component only subscribes to what it uses. A common pattern is splitting *state* and *dispatch* into two contexts, since dispatch is stable and never needs to trigger re-renders.

Third, memoize consumers with `React.memo` so that a consumer's children don't re-render unless their own props change. For very high-frequency updates where context granularity isn't enough, an external store (`useSyncExternalStore`, Zustand, Redux with selectors) lets components subscribe to slices and re-render only on the specific slice they read.

```
const value = useMemo(
  () => ({ user, setUser }),
  [user]
); // setUser is stable, so only `user` matters

<UserContext.Provider value={value}>{children}</UserContext.Provider>
```

GOTCHA

Context has no built-in selector: consumers can't subscribe to just one field of the value. Any change to the value object re-renders every consumer. If you need field-level subscriptions, split contexts or use an external store — don't reach for a giant single context.

75 Memoization Trade-offs

PERFORMANCE

Q When does memoization make things worse?

A Memoization is not free. `useMemo`, `useCallback`, and `React.memo` all add work: React must store the previous dependencies/props and compare them on every render, and it must retain the cached values in memory. When the wrapped computation or render is cheap, this bookkeeping costs more than it saves. A memoized `a + b` is strictly slower than just computing `a + b`.

It also makes code harder to read and maintain. Dependency arrays become a maintenance burden and a bug source: a missing dependency yields stale values, an extra one defeats the memo. Over-memoization spreads these arrays everywhere, and a single unstable dependency upstream can silently invalidate a whole chain of "optimized" values, so you pay the cost without the benefit.

The right approach is to treat memoization as a targeted fix for a measured problem, not a default. First reduce work structurally: move state down, lift expensive children out via `children` props, split components. Only memoize the genuinely expensive computations and the props crossing a `React.memo` boundary. The React Compiler (React 19+) aims to automate much of this, reducing manual memoization further.

```
// Net-negative: comparison > the work saved
const total = useMemo(() => price * qty, [price, qty]);

// Fine as plain code:
const total = price * qty;
```

GOTCHA

Memoizing a value passed to a child that is *not* wrapped in `React.memo` accomplishes nothing — the child re-renders with its parent regardless. Both sides of the boundary must cooperate for referential stability to pay off.

76 Hydration & Mismatches

SSR

Q What is hydration, and what causes hydration mismatches?

A Hydration is the process of attaching React to server-rendered HTML. With server-side rendering, the browser receives fully-formed markup so users see content fast. React then "hydrates" that static HTML: it renders the component tree, walks the existing DOM, and attaches event listeners and internal state without recreating the nodes. The result is an interactive app built on top of the pre-rendered markup.

For hydration to work, React expects the tree it renders on the client to match the server-rendered HTML exactly. A **hydration mismatch** happens when they differ — React logs a warning and, in React 18, falls back to client-rendering the mismatched subtree, which is slow and can cause a visible flash. Common causes: using `Date.now()`, `Math.random()`, or `window` / `localStorage` during render; browser-only branches; locale/timezone differences; or invalid HTML nesting the browser auto-corrects.

The fix is to make the initial client render deterministic and identical to the server. For genuinely client-only content, render a stable placeholder first and switch after mount (a `useEffect` -set flag), or use `suppressHydrationMismatch` / `suppressHydrationWarning` sparingly for known-different nodes like timestamps.

```
const [mounted, setMounted] = useState(false);
useEffect(() => setMounted(true), []);

// Server & first client render match; client-only
// UI appears after mount to avoid a mismatch.
return mounted ? <ClientWidget /> : null;
```

GOTCHA

React 18's `hydrateRoot` replaces `ReactDOM.hydrate`. Also, selective hydration means Suspense boundaries can hydrate independently and even prioritize the one the user interacts with first — you don't have to wait for the whole page to become interactive.

77 Render Props

PATTERNS

Q What is the render props pattern, and when is it still useful?

A A render prop is a technique for sharing logic where a component takes a *function* as a prop and calls it to determine what to render, passing it the data it manages. Instead of dictating the UI, the component owns some behavior (state, subscriptions, measurements) and delegates rendering to the caller. The name comes from a prop literally called `render`, though `children`-as-a-function is the same idea.

Before hooks, render props were the main way to share stateful, non-visual logic between components without inheritance — think mouse trackers, data fetchers, or a downshift-style autocomplete. The consumer gets maximum flexibility over markup while reusing the provider's logic. It solved the same reuse problem HOCs did, but without wrapping the component tree in extra layers.

Hooks have largely replaced render props for pure logic reuse, because a custom hook shares behavior without adding tree nesting or the "wrapper hell" render props can create. But the pattern is still alive where a library needs to hand data to caller-controlled markup — virtualization libraries, some form libraries, and headless UI components frequently use function-as-child.

```
<MouseTracker>
  {( { x, y } ) => <p>Cursor at {x}, {y}</p>}
</MouseTracker>

// MouseTracker calls props.children(state)
```

GOTCHA

An inline render-prop function is recreated every render, so a memoized child inside it can still re-render. And deeply nested render props create a "pyramid of doom." For pure logic sharing, prefer a custom hook.

78 HOCs vs Hooks

PATTERNS

Q What are Higher-Order Components, and why did hooks largely replace them?

A A Higher-Order Component is a function that takes a component and returns a new component with added behavior or props — `withRouter(Component)`, `connect(mapStateToProps)(Component)`. It's a composition pattern for reusing logic: the HOC wraps your component, injects props (data, handlers), and renders it. Before hooks, HOCs and render props were the two primary tools for sharing cross-cutting logic.

HOCs have real downsides. They add wrapper layers to the tree ("wrapper hell"), making the DevTools tree noisy and debugging harder. Injected props are implicit — it's not obvious from the component where `router` or `dispatch` came from. Prop name collisions between multiple HOCs are silent bugs. And you must manually forward refs and hoist static properties to preserve behavior.

Hooks solve the same reuse problem more cleanly: a custom hook shares stateful logic without changing the component hierarchy, makes data flow explicit (you see `const user = useUser()`), and composes linearly instead of by nesting. That's why most libraries migrated from HOCs to hooks (`connect` → `useSelector`). HOCs still appear for genuinely component-level concerns like error boundaries or wrapping third-party class components.

```
// HOC: implicit injected props, extra wrapper
const Enhanced = withUser(Profile);

// Hook: explicit, no wrapper layer
function Profile() {
  const user = useUser();
  return <span>{user.name}</span>;
}
```

GOTCHA

Never define an HOC (or call one) *inside* render — `const C = withX(Inner)` in the body creates a new component type each render, which remounts the subtree and destroys its state every time.

79 Derived-State Anti-patterns

PERFORMANCE

Q Why avoid computing derived state inside effects?

A A common anti-pattern is storing values in state and then using a `useEffect` to keep that state "in sync" with props or other state. For example, keeping `fullName` in state and updating it in an effect whenever `firstName` or `lastName` changes. This is redundant: anything you can compute from existing props/state during render should just be computed during render, not mirrored into state.

The effect approach hurts performance and correctness. It causes an extra render pass: the component renders with stale derived state, the effect runs, it sets state, and the component renders again. It also risks temporary inconsistency (the UI briefly shows the old derived value) and adds a dependency array that's easy to get wrong. React's guidance is explicit: "you don't need an effect to transform data for rendering."

Instead, calculate the value inline during render. If the calculation is genuinely expensive, wrap it in `useMemo` — still no extra render, still no duplicated source of truth. Reserve effects for synchronizing with *external* systems (network, subscriptions, DOM), not for reacting to your own state.

```
// Anti-pattern: extra render + duplicated state
const [full, setFull] = useState('');
useEffect(() => setFull(first + ' ' + last), [first, last]);

// Better: derive during render
const full = first + ' ' + last;
```

GOTCHA

If you need to *reset* state when a prop changes, don't do it in an effect either — pass a `key` to remount, or adjust state during render with the "set state while rendering" pattern. Effects are the slowest, buggiest option for both cases.

80 Initial Load Performance

PERFORMANCE

Q How do you measure and improve initial load performance?

A Initial load is dominated by how much JavaScript the browser must download, parse, and execute before the app is interactive. Measure it with real metrics: Lighthouse and the web-vitals library for Largest Contentful Paint, Total Blocking Time, and Time to Interactive; and a bundle analyzer (`webpack-bundle-analyzer` , `rollup-plugin-visualizer`) to see what's actually in each chunk. You can't fix a bundle you haven't measured.

The biggest levers are code splitting and tree-shaking. Split at the route level with `React.lazy` so users only download the page they land on. Ensure your bundler can tree-shake — that requires ES modules (not CommonJS), `"sideEffects": false` in `package.json` where accurate, and importing only what you use (`import { debounce } from 'lodash-es'` instead of the whole library). Audit heavy dependencies and replace or defer them.

Other wins: compress with gzip/Brotli, serve modern bundles to modern browsers, defer non-critical work, and consider server-side rendering or static generation so users see content before the JS loads. Preload critical chunks and lazy-load below-the-fold or interaction-triggered features. Always re-measure after each change to confirm impact.

```
// Route-level splitting shrinks the initial bundle
const Dashboard = React.lazy(() => import('./Dashboard'));
const Settings = React.lazy(() => import('./Settings'));

// Import members so unused code is tree-shaken
import { debounce } from 'lodash-es';
```


GOTCHA

Tree-shaking silently fails on CommonJS packages and on modules with side effects. A single `import 'whole-library'` or a barrel file re-exporting everything can pull in far more than you use — verify with the analyzer, don't assume.

81 Suspense for Lazy & Data

ADVANCED

Q What is Suspense, and how does it work for both lazy components and data fetching?

A Suspense is a mechanism that lets a component "suspend" rendering while it waits for something asynchronous, showing a declarative fallback UI in the meantime. The WHY: without Suspense, every component that loads something async needs its own `isLoading` state and conditional JSX, scattering loading logic everywhere. Suspense lifts that concern up to a boundary, so a parent declares the fallback once and any descendant can suspend.

The oldest use is `React.lazy`. A lazy component returns a promise for its code chunk; while that chunk downloads, the nearest **Suspense** boundary above it renders the `fallback`. This is how you code-split routes without manual loading flags.

Suspense for *data* is the newer, more powerful case. A data source (a framework loader, or the React 19 `use()` hook) throws a promise; React catches it, shows the fallback, and retries the render when the promise resolves. This enables render-as-you-fetch and streaming SSR, where the server sends HTML for ready parts and streams the rest as data arrives.

```
const Profile = lazy(() => import('./Profile'));

function App() {
  return (
    <Suspense fallback={<Spinner />}>
      <Profile />
    </Suspense>
  );
}
```

GOTCHA

Suspense only catches promises thrown during render, not inside event handlers or effects. You cannot make a bare `useEffect` + `fetch` suspend; you need a Suspense-integrated data source. Also, a boundary re-shows its fallback on any suspending update unless you wrap the update in a transition.

82 React Server Components

ADVANCED

Q What are React Server Components, and how do server and client components differ?

A React Server Components (RSC) are components that run only on the server and never ship their code to the browser. The WHY: much UI is just reading data and producing markup — it does not need interactivity, and sending its JavaScript to the client is pure waste. RSC lets you render that work on the server, send a compact serialized description of the output, and keep the bundle small.

A server component can be `async`, await a database query or file read directly, and access server-only secrets — because its code never leaves the server. What it cannot do is use state, effects, or browser event handlers, since there is no client runtime for it.

A client component is the traditional React component, marked with the `'use client'` directive at the top of the file. It ships to the browser and can use `useState`, `useEffect`, and `onClick`. Server components can import and render client components (passing serializable props), composing the two into one tree.

```
// Server Component (default in RSC frameworks)
async function Page() {
  const posts = await db.posts.findAll();
  return <PostList posts={posts} />;
}

// Client Component
'use client';
function LikeButton() {
  const [liked, setLiked] = useState(false);
}
```

GOTCHA

The `'use client'` directive marks the *boundary*, not just one component — everything imported into a client module becomes part of the client bundle. Props passed from server to client components must be serializable, so you cannot pass a function (except server actions) or a class instance across the boundary.

83 Server Components vs SSR

ADVANCED

Q How do Server Components differ from server-side rendering (SSR)?

A They solve different problems and are not alternatives — you often use both. The WHY for the confusion: both "run React on the server," but SSR is about *when* the initial HTML is produced, while RSC is about *where* a component's code lives and whether it ships to the client at all.

Traditional SSR takes your normal component tree, renders it to an HTML string on the server for a fast first paint, sends it, and then *hydrates* — the full JavaScript bundle for every component still downloads to the browser and re-attaches. SSR improves perceived load and SEO, but the client bundle is unchanged.

Server Components change what gets bundled. A server component's code never reaches the browser; only its rendered output (as a serialized payload) and the code for actual client components are sent. So RSC reduces bundle size and lets you fetch data at the component level on the server, while SSR is purely a rendering-timing strategy.

In practice a framework like Next.js renders server components on the server, produces streaming HTML via SSR for the client components within them, and then hydrates only those client parts. RSC and SSR compose: RSC decides what code ships, SSR decides how the first HTML is delivered.

GOTCHA

Interviewers love "is RSC just SSR?" — the crisp answer is no: SSR still ships every component's JS to hydrate; RSC omits server-only component code from the bundle entirely. A component can be a server component that is also SSR-rendered, but the two concepts are orthogonal.

84 The use() Hook

ADVANCED

Q What is the React 19 use() hook and how is it different from other hooks?

A The `use()` API reads the value of a resource — a promise or a context — during render. The WHY: before it, reading a promise required Suspense-integrated libraries, and reading context required the rules-of-hooks-bound `useContext`. `use()` unifies these and, crucially, relaxes the rules: it may be called conditionally and inside loops.

When you pass a promise to `use()`, the component suspends until the promise resolves, then `use()` returns the resolved value; if it rejects, the nearest error boundary catches it. This makes render-as-you-fetch ergonomic and integrates naturally with the nearest Suspense boundary.

Unlike other hooks, `use()` is not bound by the "call at the top level only" rule — it can appear after an early return or inside an `if`. This is why it is called an API rather than strictly a hook, though it still must be called during render.

```
function Comments({ commentsPromise }) {  
  // suspends until resolved (React 19)  
  const comments = use(commentsPromise);  
  return comments.map((c) => <p key={c.id}>{c.text}</p>);  
}
```

GOTCHA

Do not create the promise inside the component that calls `use()` during render — a new promise each render causes an infinite suspend/re-render loop. Create promises in a server component, a cached loader, or an event handler and pass them down.

85 Actions & useActionState

ADVANCED

Q What are Actions in React 19, and how does useActionState work?

A Actions are async functions React manages for you, tracking pending state, errors, and optimistic updates around a data mutation. The WHY: submitting a form previously meant hand-rolling `isPending`, try/catch error state, and resetting — boilerplate repeated everywhere. Actions bake this lifecycle into React and integrate with forms and transitions.

`useActionState` wraps an action function and returns the current state, a wrapped action to dispatch, and a `isPending` boolean. You give it a reducer-like function `(prevState, formData) => newState` and an initial state; React runs it inside a transition so the UI stays responsive.

Passing the wrapped action to a form's `action` prop wires submission automatically — React calls your function with the form data, marks it pending during the await, and updates state with the return value. This works with progressive enhancement in RSC frameworks.

```
const [error, submitAction, isPending] = useActionState(
  async (prev, formData) => {
    const res = await updateName(formData.get('name'));
    if (res.error) return res.error;
    return null;
  },
  null
);
// <form action={submitAction}>
```

GOTCHA

The first argument to your action is the *previous state*, not the form data — a common mix-up. Also `useActionState` was briefly named `useFormState` in canaries and lived in `react-dom`; in React 19 it is `useActionState` from `react`.

86 useOptimistic

ADVANCED

Q What problem does useOptimistic solve, and how do you use it?

A useOptimistic lets you show a provisional "optimistic" state immediately while an async action is still in flight, then automatically fall back to the real state when it settles. The WHY: users expect instant feedback — a sent message should appear at once — but the server round-trip takes time, and manually reconciling the temporary UI with the confirmed result is error-prone.

You pass it the actual state and a function that merges an optimistic value into it. It returns the optimistic state to render plus a setter you call inside your action. React shows the optimistic value during the pending transition and, once the action completes and the real state updates, discards the optimistic layer and re-renders with the true value.

The elegance is the automatic revert: you never manually roll back. If the action fails, the optimistic value simply disappears when React re-renders from the confirmed state, so the UI self-corrects.

```
const [optimisticMsgs, addOptimistic] = useOptimistic(
  messages,
  (state, newMsg) => [...state, { text: newMsg, sending: true }]
);

async function send(formData) {
  addOptimistic(formData.get('msg')); // shows instantly
  await sendMessage(formData);       // then reconciles
}
```

GOTCHA

Optimistic updates only persist while the surrounding transition/action is pending. Call `addOptimistic` inside an action (or a function wrapped by a transition), not in a raw handler with no pending work, or the optimistic state reverts immediately.

87 Portals

ADVANCED

Q What is `createPortal` and when do you need it?

A A portal renders children into a different DOM node than the parent component's own DOM position, while keeping them in the same React tree. The WHY: sometimes CSS constraints — `overflow: hidden`, `z-index` stacking contexts, or `transform` on an ancestor — clip or mis-layer content like modals and tooltips. A portal lets you visually escape the parent container by mounting into `document.body` while preserving React context and event bubbling.

Crucially, a portal only changes the physical DOM placement, not the logical React tree. State, context, and even event propagation still flow through the React parent — a click inside a portaled modal bubbles to handlers on its React ancestor, even though it lives elsewhere in the DOM.

Typical use cases: modals, dialogs, dropdowns, tooltips, and toast notifications — anything that must render above or outside its parent's clipping/stacking context.

```
import { createPortal } from 'react-dom';

function Modal({ children }) {
  return createPortal(
    <div className="overlay">{children}</div>,
    document.body
  );
}
```

GOTCHA

Because events still bubble through the React tree, a click inside a portaled dropdown can trigger an "outside click" handler on a React ancestor even though the DOM node is elsewhere. Account for React-tree bubbling, not just DOM position, when writing dismiss logic.

88 Refs & forwardRef

ADVANCED

Q How do refs and forwardRef work, and what changed in React 19?

A A ref is a mutable container (`{ current: ... }`) whose changes do not trigger re-renders. Its two main jobs are holding a reference to a DOM node and storing a mutable value that should survive renders without causing them. The WHY for refs over state: some values (a timer id, a DOM element) are not rendering inputs, so making them state would cause needless re-renders.

Historically, function components could not receive a `ref` prop directly, so `forwardRef` wrapped a component to explicitly forward a ref to a child DOM node — letting a parent grab, say, the underlying `<input>` of a custom component.

React 19 makes `ref` an ordinary prop for function components, so `forwardRef` is no longer needed in most cases — you just destructure `ref` from props. `forwardRef` still works for backward compatibility but is being deprecated in favor of the direct prop.

```
// React 19: ref as a regular prop
function Input({ ref, ...props }) {
  return <input ref={ref} {...props} />;
}

// Pre-19: needed forwardRef
const Input = forwardRef((props, ref) =>
  <input ref={ref} {...props} />);
```

GOTCHA

Mutating `ref.current` does not re-render, so never read a ref value to decide what to render — the UI will be stale. Also do not read or write `ref.current` during render (except lazy initialization); do it in effects or handlers.

89 Effects vs Event Handlers

ADVANCED

Q When do you need an Effect, and when might you not?

A Effects are for synchronizing your component with an *external system* — a subscription, a network connection, a non-React widget, the document title. The WHY this matters: developers overuse effects for logic that belongs elsewhere, causing extra renders, race conditions, and bugs. The React docs literally have a page titled "You Might Not Need an Effect."

The key distinction: code that runs *because a specific interaction happened* belongs in an event handler; code that runs *because the component is displayed and needs to stay in sync with something external* belongs in an effect. Submitting a form on click is an event; opening a chat connection while a room is shown is an effect.

Two common anti-patterns: (1) using an effect to transform data for rendering — instead compute it during render, memoizing if expensive; (2) using an effect to reset or adjust state when a prop changes — often better handled with a `key` to remount, or by computing derived values inline.

```
// Anti-pattern: effect to derive state
useEffect(() => { setFullName(first + ' ' + last); }, [first, last]);

// Better: derive during render
const fullName = first + ' ' + last;
```

GOTCHA

If an effect has no external system and only reacts to a user action, it is probably misplaced — move it into the handler. A tell-tale smell is an effect whose only job is to call `setState` based on other state or props.

90 React Router Basics

ADVANCED

Q How does client-side routing work with React Router?

A Client-side routing swaps which components render based on the URL without a full page reload. The WHY: a single-page app loads once, then intercepts navigation to update the view via the History API, keeping state and avoiding server round-trips for each page — giving app-like transitions.

React Router provides a router (e.g. `BrowserRouter` or a data router via `createBrowserRouter`), `Routes` / `Route` to map URL paths to elements, and `Link` / `NavLink` to navigate without reloading. Routes can nest, and an `Outlet` renders the matched child inside a shared layout.

Dynamic segments like `/users/:id` are read with `useParams`; programmatic navigation uses `useNavigate`. Modern data routers add `loader` functions to fetch data before a route renders and `action` functions to handle form submissions.

```
import { Routes, Route, Link, useParams } from 'react-router-dom';

<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/users/:id" element={<User />} />
</Routes>
// const { id } = useParams();
```

GOTCHA

Use `Link`, not a plain `<a href>`, for internal navigation — a raw anchor triggers a full page reload, discarding app state. Reserve `<a>` for external URLs.

91 Accessibility in React

ADVANCED

Q How do you build accessible React components?

A Accessibility (a11y) means the UI works for assistive technology and keyboard users. The WHY: React outputs the same DOM as any web app, so the same rules apply — but because we build custom widgets from generic `div`s, it is easy to lose the semantics browsers give for free. Starting from semantic HTML is the single biggest lever.

Prefer real elements: a `<button>` over a clickable `<div>` gets focusability, Enter/Space activation, and the right role automatically. Use `<label>` tied to inputs, headings in order, and lists for lists. In JSX, remember it is `htmlFor` (not `for`) and `className`.

When you must build custom widgets, add ARIA roles and attributes to fill the semantic gap — `aria-label`, `aria-expanded`, `role` — but ARIA is a last resort, not a substitute for the right element. Manage focus explicitly for modals and route changes (move focus in, trap it, restore it on close) using refs.

```
// Prefer semantics; add ARIA only to fill gaps
<button aria-expanded={open} onClick={toggle}>
  Menu
</button>
<label htmlFor="email">Email</label>
<input id="email" />
```

GOTCHA

A `<div onClick>` is not keyboard-accessible — it cannot be focused or activated with Enter/Space and exposes no role. The first rule of ARIA is "don't use ARIA": reach for the native element first.

92 React with TypeScript

ADVANCED

Q How do you type props, state, and events in React with TypeScript?

A TypeScript adds compile-time safety to React by describing the shapes of props, state, and event objects. The WHY: React's data flow is all about passing typed values through props and callbacks, so a type system catches missing props, wrong shapes, and incorrect event handling before runtime — the most common class of React bugs.

Type props with an `interface` or `type` and annotate the component's parameter. For state, `useState` usually infers from the initial value, but you specify the generic when the initial value is `null` or a union, e.g. `useState<User | null>(null)`.

Event handlers use React's synthetic event types, like

`React.ChangeEvent<HTMLInputElement>` or `React.MouseEvent<HTMLButtonElement>`.

For children, use `React.ReactNode`. Prefer typing the handler's parameter over the whole function so inference stays clean.

```
interface Props {
  label: string;
  onSave: (id: number) => void;
}

function Field({ label, onSave }: Props) {
  const [value, setValue] = useState<string>('');
  const onChange = (e: React.ChangeEvent<HTMLInputElement>) =>
    setValue(e.target.value);
}
```

GOTCHA

Avoid `React.FC` for typing components if you want implicit children control — it was criticized for silently allowing `children` and complicating generics. Prefer typing props directly on the parameter.

93 Testing Philosophy (RTL)

ADVANCED

Q What is the testing philosophy behind React Testing Library?

A React Testing Library (RTL) is built on one principle: "the more your tests resemble the way your software is used, the more confidence they give you." The WHY: tests coupled to implementation details — internal state, method names, component structure — break on every refactor even when behavior is unchanged, and they pass even when the user-facing behavior is broken. RTL steers you to test behavior instead.

Concretely, you query the rendered output the way a user perceives it — by visible text, labels, and roles — and you interact the way a user does — clicking, typing. You avoid reaching into the instance to read state or call methods, which RTL deliberately makes hard.

The payoff is refactor-resistant tests: rename a state variable, switch from a class to hooks, or restructure the JSX, and a behavior-focused test keeps passing as long as the user sees the same thing. This makes the test suite a safety net rather than a maintenance burden.

```
// Test what the user sees, not internals
render(<Counter />);
await userEvent.click(screen.getByRole('button', { name: /increment/i }));
expect(screen.getByText('Count: 1')).toBeInTheDocument();
```

GOTCHA

If a test asserts on internal state or a component's private methods, it is testing implementation, not behavior — a red flag. A good heuristic: could a user observe what you are asserting? If not, reconsider the assertion.

94 RTL Queries & user-event

ADVANCED

Q How do RTL queries work, and why prefer user-event over fireEvent?

A RTL queries come in three families with different behaviors: `getBy` throws if not found (for elements that should exist), `queryBy` returns null (for asserting absence), and `findBy` returns a promise that retries (for elements appearing asynchronously). The WHY: choosing the right variant makes the intent explicit and gives good failure messages.

There is also a priority order for *which* query to use. Prefer accessible queries — `getByRole`, `getByLabelText`, `getByText` — because they mirror how users and assistive tech find things. Fall back to `getByTestId` only when nothing accessible works, since test ids are invisible to users.

`user-event` is preferred over `fireEvent` because it simulates full interactions the way a real user triggers them: typing dispatches `keydown`, `keypress`, `input`, and `keyup`, and clicking checks that the element is not disabled and fires the correct sequence. `fireEvent` dispatches a single raw event, missing the surrounding reality.

```
const input = screen.getByLabelText('Email');
await userEvent.type(input, 'a@b.com'); // realistic keystrokes

const toast = await screen.findByText(/saved/i); // waits for async
expect(screen.queryByRole('alert')).not.toBeInTheDocument();
```

GOTCHA

Modern `user-event` (v14+) is async — every interaction returns a promise you must `await`. Forgetting the `await` causes assertions to run before the UI updates, producing flaky or false-passing tests.

95 Testing Hooks & Async UI

ADVANCED

Q How do you test custom hooks and asynchronous UI updates?

A Custom hooks cannot be called outside a component, so you test them with `renderHook` from React Testing Library, which mounts a tiny host component and returns the hook's result in `result.current`. The WHY: this lets you exercise a hook's logic directly without inventing a throwaway wrapper, while still running it inside React's real rendering.

To trigger updates, wrap state-changing calls in `act` (or use the async `act`) so React flushes effects and re-renders before you assert. After an update, read the new value from `result.current` again — it points to the latest render.

For async UI, prefer `findBy*` queries or `waitFor`, which poll until the expectation passes or times out, instead of arbitrary timeouts. This handles data that arrives after a promise resolves without hardcoding delays.

```
const { result } = renderHook(() => useCounter());
act(() => result.current.increment());
expect(result.current.count).toBe(1);

// async UI
render(<Profile />);
expect(await screen.findByText('Ada')).toBeInTheDocument();
```

GOTCHA

An "not wrapped in act(...)" warning usually means a state update happened after the test finished — often an un-awaited async update. Use `findBy` / `waitFor` to await the settled state rather than ignoring the warning.

96 Mocking in Tests

ADVANCED

Q How do you mock modules and network requests in React tests?

A Mocking replaces a real dependency with a controlled fake so tests are fast, deterministic, and isolated from the network. The WHY: hitting a real API makes tests slow, flaky, and dependent on external state; mocking lets you assert how your component behaves given specific responses, including errors.

For module mocking, Jest's `jest.mock('./api')` (or Vitest's `vi.mock`) swaps a module with a fake whose functions you control via `mockResolvedValue` or `mockRejectedValue`. This is good for unit-level isolation of a specific dependency.

For network requests, the modern best practice is Mock Service Worker (MSW), which intercepts requests at the network layer rather than stubbing `fetch`. Because it mocks the actual HTTP boundary, your component and data-fetching code run unchanged, giving higher-confidence tests that survive refactors of the fetch implementation.

```
// Module mock
jest.mock('./api');
api.getUser.mockResolvedValue({ name: 'Ada' });

// MSW handler (network-level)
http.get('/api/user', () =>
  HttpResponse.json({ name: 'Ada' }));
```

GOTCHA

Reset mocks between tests (`jest.clearAllMocks` or `restoreHandlers`) or state leaks across tests and causes order-dependent failures. Prefer MSW over stubbing `global.fetch` so tests survive switching to axios or React Query.

97 Common Anti-Patterns

ADVANCED

Q What are common React anti-patterns to avoid?

A The recurring theme in React anti-patterns is fighting the framework's data flow instead of working with it. The WHY: React expects unidirectional data flow, pure renders, and state as the single source of truth; violating those produces stale UI, unnecessary renders, and hard-to-trace bugs.

Frequent offenders: mutating state directly instead of producing a new value (React compares by reference and skips the update); duplicating props into state, which then goes stale when the prop changes; and overusing effects to sync state that could be derived during render.

Others: defining components inside another component's body (remounts and loses state every render); doing side effects during render (render must be pure); prop-drilling many levels instead of using context or composition; and premature memoization that adds complexity without measured benefit.

```
// Anti-pattern: mutate then set
state.items.push(x); setItems(state.items); // same ref, no update

// Correct: new reference
setItems([...state.items, x]);
```

GOTCHA

Defining a component inside another component looks harmless but recreates the type every render, so React unmounts and remounts the subtree each time — losing its state and effects. Hoist components to module scope.

98 Index-as-Key Anti-Pattern

ADVANCED

Q Why is using the array index as a key an anti-pattern?

A Keys tell React's reconciler which list items are the same across renders so it can match, reuse, and reorder elements correctly. The WHY the index is dangerous: when the list can reorder, insert, or delete, an item's index changes, so React associates a DOM node and its internal state with the wrong data — producing subtle bugs.

The classic symptom: a list of inputs keyed by index. Delete the first row, and every remaining row's index shifts down by one, so React thinks row 1 is still row 1 and keeps its DOM node — but now it holds the next item's data, while the typed input value stays attached to the wrong entry. Checkbox and focus state leak between rows the same way.

The fix is a stable, unique identifier tied to the data itself — a database id or a generated uuid created when the item is added. Index keys are acceptable only for static lists that never reorder, filter, or change length.

```
// Risky: index as key on a mutable list
{items.map((it, i) => <Row key={i} item={it} />)}

// Safe: stable id
{items.map((it) => <Row key={it.id} item={it} />)}
```

GOTCHA

Do not paper over it with `key={Math.random()}` — a new key every render forces React to destroy and recreate every item, killing performance and blowing away state and focus. Keys must be stable *and* unique.

99 Lifecycle Methods to Hooks

ADVANCED

Q How do class lifecycle methods map to hooks?

A When reading legacy class code, it helps to translate lifecycle methods into their hook equivalents. The WHY: hooks reorganize logic by *concern* rather than by *timing* — a single `useEffect` can express setup and teardown together, whereas classes split related logic across `componentDidMount` and `componentWillUnmount`.

`componentDidMount` maps to `useEffect(fn, [])` — runs once after mount.

`componentDidUpdate` maps to `useEffect(fn, [deps])` that runs when a dependency changes. `componentWillUnmount` maps to the cleanup function returned from an effect. Together, one effect covers the full mount/update/unmount cycle for a given concern.

Other mappings: `this.state` / `setState` become `useState` or `useReducer`; `shouldComponentUpdate` becomes `React.memo`; expensive per-render computation becomes `useMemo`. There is no direct hook for `getSnapshotBeforeUpdate` or error boundaries — error boundaries still require a class.

```
// componentDidMount + componentWillUnmount
useEffect(() => {
  const id = setInterval(tick, 1000);
  return () => clearInterval(id); // cleanup = unmount
}, []);
```

GOTCHA

An effect is not a perfect 1:1 for a lifecycle method — it runs after paint and re-runs whenever dependencies change, so thinking "this is just `componentDidUpdate`" can mislead. Think in terms of synchronization, not timing.

100 Reasoning About Unknowns

ADVANCED

Q How should you reason about a React interview question you have never seen?

A The meta-skill is reducing any unfamiliar question to a few durable first principles rather than recalling a memorized answer. The WHY: React's surface area keeps growing, but it rests on a small core — UI is a pure function of state, data flows down, and effects synchronize with the outside world. Almost every "why" traces back to these.

A reliable framework: (1) state the core mental model — `UI = f(state)`; (2) ask what triggers a re-render — a state or prop change, or a parent re-rendering; (3) ask about identity and equality — React compares by reference, which explains keys, memoization, and dependency arrays; (4) separate render (must be pure) from effects (side effects, cleanup); (5) consider the boundary — what is React's job versus the external system's.

When stuck, reason out loud from these principles and connect the specific API to the model. Interviewers value seeing you derive an answer from fundamentals over reciting one — it demonstrates you can handle the API you have not memorized.

Finally, tie behavior to the "why": explain that immutability enables cheap reference comparison, that keys enable correct reconciliation, that effects run after render because render must stay pure. Grounding answers in causes, not rules, is what distinguishes a senior response.

GOTCHA

Do not bluff a specific API name you are unsure of. It is stronger to say "I would reach for something that synchronizes with an external system here — an effect with cleanup" and reason from the model than to confidently cite a hook that does not exist.

You made it to 100.

If you can explain most of these out loud, you are ready for almost any React interview. The strongest candidates are not the ones who memorized hook signatures, they are the ones who can reason about why React re-renders and what a pattern costs. When you hit a question that is not here, walk it the same way: what problem does this solve, and what is the trade-off?

Found this useful?

This is part of a series of deep-dive interview guides. Follow for the next one, a new technology every few days.

Save this PDF and share it with someone who is interviewing.